Stock Prediction

Buy / Hold / Sell: XGBoost (tuned) + LSTM + GRU | Medium-term 21-day horizon

Install & Imports

```
!pip install yfinance xgboost scikit-learn pandas numpy matplotlib
seaborn backtesting optuna torch torchvision -q

import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

import yfinance as yf
from xgboost import XGBClassifier
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)
from sklearn.model_selection import TimeSeriesSplit
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
from sklearn.utils.class_weight import compute_sample_weight
from backtesting import Backtest, Strategy

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'All imports OK, Device: {DEVICE}')
```

```
All imports OK, Device: cpu
```

#Configuration

```
MODEL_NAME = 'xgboost'

TICKERS = [
    'AAPL', 'MSFT', 'GOOGL', 'AMZN', 'META',
    'JPM',  'BAC',  'GS',
    'XOM',  'CVX',
```

```
    'JNJ',  'PFE',
    'WMT',  'HD',
    'TSLA', 'NVDA',
]

#Date range
START_DATE = '2010-01-01'
END_DATE   = '2025-12-31'

#Label thresholds
HORIZON        = 21      #days forward to look
BUY_THRESHOLD  =  0.05   #gain > 5%
SELL_THRESHOLD = -0.04   #drop > 4%
MAX_DRAWDOWN   = -0.08   #reject BUY if drops >8% along the way, too
much swing

#Cross-validation
TEST_FRACTION = 0.2    #test last 20% of data
N_CV_SPLITS   = 5      #Split folds
CV_GAP        = 21     #gap between train/val to prevent leakage

print(f'Model: {MODEL_NAME} \nTickers: {len(TICKERS)} \nFrom:
{START_DATE} to {END_DATE}')
```

```
Model: xgboost
Tickers: 16
From: 2010-01-01 to 2025-12-31
```

#Download Price Data

```
def download_data(tickers, start, end):
    print(f'Downloading {len(TICKERS)} tickers from: {START_DATE} to
{END_DATE}')
    stock_data = {}

    for ticker in tickers:
        try:
            df = yf.download(ticker, start=start, end=end,
                             auto_adjust=True, progress=False)
            if len(df) > 300:
                df.columns = [c[0] if isinstance(c, tuple) else c for
c in df.columns]
                stock_data[ticker] = df
                print(f'  (ok) {ticker}: {len(df)} rows')
            else:
                print(f'  (skip) {ticker}: too few rows, skipped')
        except Exception as e:
            print(f'  (fail) {ticker}: {e}')

    print('\nFetching Benchmarks...')
```

```
    spy = yf.download('SPY',  start=start, end=end, auto_adjust=True,
progress=False)
    vix = yf.download('^VIX', start=start, end=end, auto_adjust=True,
progress=False)
    spy.columns = [c[0] if isinstance(c, tuple) else c for c in
spy.columns]
    vix.columns = [c[0] if isinstance(c, tuple) else c for c in
vix.columns]
    print(f"(done) SPY: {len(spy)} rows, VIX: {len(vix)} rows\n")

    return stock_data, spy, vix

stock_data, spy, vix = download_data(TICKERS, START_DATE, END_DATE)
```

```
Downloading 16 tickers from: 2010-01-01 to 2025-12-31
  (ok) AAPL: 4023 rows
  (ok) MSFT: 4023 rows
  (ok) GOOGL: 4023 rows
  (ok) AMZN: 4023 rows
  (ok) META: 3424 rows
  (ok) JPM: 4023 rows
  (ok) BAC: 4023 rows
  (ok) GS: 4023 rows
  (ok) XOM: 4023 rows
  (ok) CVX: 4023 rows
  (ok) JNJ: 4023 rows
  (ok) PFE: 4023 rows
  (ok) WMT: 4023 rows
  (ok) HD: 4023 rows
  (ok) TSLA: 3901 rows
  (ok) NVDA: 4023 rows

Fetching Benchmarks...
(done) SPY: 4023 rows, VIX: 4023 rows
```

Feature Engineering

```
FEATURE_COLS = [
    'ret_1d', 'ret_5d', 'ret_21d', 'ret_63d',
    'vol_10', 'vol_21', 'vol_63', 'vol_ratio',
    'price_vs_sma20', 'price_vs_sma50', 'price_vs_sma200',
    'sma20_vs_sma50', 'sma50_vs_sma200',
    'rsi14', 'rsi14_change',
    'macd_norm', 'macd_hist',
    'bb_pos', 'bb_width',
    'atr14_norm',
    'range_pct', 'gap_pct', 'close_pos',
```

```
    'vol_ratio_5d', 'vol_ratio_21d', 'obv_ratio',
    'spy_above_200ma', 'spy_ret_21d',
    'vix_level', 'vix_5d_change', 'vix_vs_sma20',
    'rs_vs_spy_21d', 'rs_vs_spy_5d',
]

def build_features(df, spy, vix):
    df = df.copy()

    # Returns
    df['ret_1d']  = df['Close'].pct_change(1)
    df['ret_5d']  = df['Close'].pct_change(5)
    df['ret_21d'] = df['Close'].pct_change(21)
    df['ret_63d'] = df['Close'].pct_change(63)

    # Volatility
    df['vol_10']    = df['ret_1d'].rolling(10).std()
    df['vol_21']    = df['ret_1d'].rolling(21).std()
    df['vol_63']    = df['ret_1d'].rolling(63).std()
    df['vol_ratio'] = df['vol_10'] / (df['vol_63'] + 1e-9)

    # Moving averages as ratios (scale-invariant)
    sma20  = df['Close'].rolling(20).mean()
    sma50  = df['Close'].rolling(50).mean()
    sma200 = df['Close'].rolling(200).mean()
    df['price_vs_sma20']  = df['Close'] / sma20  - 1
    df['price_vs_sma50']  = df['Close'] / sma50  - 1
    df['price_vs_sma200'] = df['Close'] / sma200 - 1
    df['sma20_vs_sma50']  = sma20 / sma50  - 1
    df['sma50_vs_sma200'] = sma50 / sma200 - 1

    # RSI(14) — Wilder smoothing
    delta    = df['Close'].diff()
    avg_gain = delta.clip(lower=0).ewm(com=13, adjust=False).mean()
    avg_loss = (-delta.clip(upper=0)).ewm(com=13, adjust=False).mean()
    df['rsi14']        = 100 - (100 / (1 + avg_gain / (avg_loss + 1e-
9)))
    df['rsi14_change'] = df['rsi14'].diff(5)

    # MACD (12,26,9)
    ema12 = df['Close'].ewm(span=12, adjust=False).mean()
    ema26 = df['Close'].ewm(span=26, adjust=False).mean()
    macd  = ema12 - ema26
    df['macd_norm'] = macd / df['Close']
    df['macd_hist'] = macd - macd.ewm(span=9, adjust=False).mean()

    # Bollinger Bands
    bb_mid   = df['Close'].rolling(20).mean()
    bb_std   = df['Close'].rolling(20).std()
    bb_upper = bb_mid + 2 * bb_std
```

```
    bb_lower = bb_mid - 2 * bb_std
    df['bb_pos']   = (df['Close'] - bb_lower) / (bb_upper - bb_lower +
1e-9)
    df['bb_width'] = (bb_upper - bb_lower) / (bb_mid + 1e-9)

    # ATR(14)
    prev_close = df['Close'].shift(1)
    tr = pd.concat([
        df['High'] - df['Low'],
        (df['High'] - prev_close).abs(),
        (df['Low']  - prev_close).abs()
    ], axis=1).max(axis=1)
    df['atr14_norm'] = tr.rolling(14).mean() / df['Close']

    # Price patterns
    df['range_pct'] = (df['High'] - df['Low']) / df['Close']
    df['gap_pct']   = (df['Open'] - prev_close) / (prev_close + 1e-9)
    df['close_pos'] = (df['Close'] - df['Low']) / (df['High'] -
df['Low'] + 1e-9)

    # Volume
    if 'Volume' in df.columns:
        vol_ma20 = df['Volume'].rolling(20).mean()
        df['vol_ratio_5d']  = df['Volume'].rolling(5).mean()  /
(vol_ma20 + 1)
        df['vol_ratio_21d'] = df['Volume'].rolling(21).mean() /
(vol_ma20 + 1)
        obv = (np.sign(df['ret_1d']) * df['Volume']).cumsum()
        df['obv_ratio'] = obv / (obv.rolling(21).mean() + 1e-9)

    # Market regime
    spy_c = spy['Close'].reindex(df.index, method='ffill')
    vix_c = vix['Close'].reindex(df.index, method='ffill')
    df['spy_above_200ma'] = (spy_c >
spy_c.rolling(200).mean()).astype(int)
    df['spy_ret_21d']     = spy_c.pct_change(21)
    df['vix_level']       = vix_c
    df['vix_5d_change']   = vix_c.pct_change(5)
    df['vix_vs_sma20']    = vix_c / (vix_c.rolling(20).mean() + 1e-9)
- 1

    # Relative strength vs market
    df['rs_vs_spy_21d'] = df['ret_21d'] - df['spy_ret_21d']
    df['rs_vs_spy_5d']  = df['ret_5d']  - spy_c.pct_change(5)

    return df

print(f"build_features defined: {len(FEATURE_COLS)} metrics ready")
```

```
build_features defined: 33 metrics ready
```

Label Creation

```
def create_labels(df, horizon=21, buy_thresh=0.05, sell_thresh=-0.04,
max_dd=-0.08):
    df = df.copy()
    close = df['Close'].values
    n = len(close)

    future_returns = np.full(n, np.nan)
    max_drawdowns = np.full(n, np.nan)

    for i in range(n - horizon):
        window = close[i+1 : i+horizon+1]
        future_returns[i] = (window[-1] - close[i]) / close[i]
        max_drawdowns[i] = (window.min() - close[i]) / close[i]

    df['future_return'] = future_returns
    df['max_drawdown_fwd'] = max_drawdowns
    df['label'] = 1   # default HOLD

    buy_mask = (df['future_return'] >  buy_thresh) &
(df['max_drawdown_fwd'] > max_dd)
    sell_mask =  df['future_return'] < sell_thresh
    df.loc[buy_mask,  'label'] = 2
    df.loc[sell_mask, 'label'] = 0

    return df

print('Labels created')
```

```
Labels created
```

Build Combined Dataset

```
def build_dataset(stock_data, spy, vix,
                  horizon=HORIZON,
                  buy_thresh=BUY_THRESHOLD,
                  sell_thresh=SELL_THRESHOLD,
                  max_dd=MAX_DRAWDOWN):
    frames = []

    for ticker, raw_df in stock_data.items():
        try:
            df = build_features(raw_df, spy, vix)
            df = create_labels(df, horizon=horizon,
                               buy_thresh=buy_thresh,
                               sell_thresh=sell_thresh,
                               max_dd=max_dd)
```

```
            available = [c for c in FEATURE_COLS if c in df.columns]
            df = df.dropna(subset=available + ['label']).copy()
            df['ticker'] = ticker
            frames.append(df[available + ['label', 'ticker']])
        except Exception as e:
            print(f'  Error on {ticker}: {e}')

    combined = pd.concat(frames).sort_index()
    print(f'Combined dataset: {len(combined):,} rows,
{combined["ticker"].nunique()} tickers')
    print('\nLabel distribution:')
    dist = combined['label'].value_counts(normalize=True).sort_index()
    dist.index = dist.index.map({0:'SELL', 1:'HOLD', 2:'BUY'})
    print(dist.round(3).to_string())
    return combined

combined = build_dataset(stock_data, spy, vix)
```

```
Combined dataset: 60,463 rows, 16 tickers

Label distribution:
label
SELL    0.199
HOLD    0.507
BUY     0.294
```

#XGBoost Model

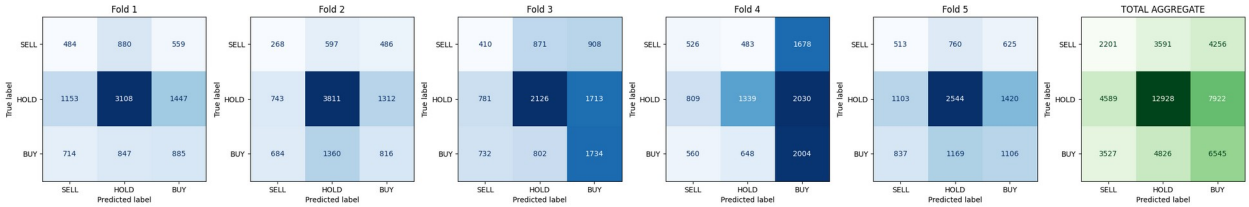
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import TimeSeriesSplit
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_sample_weight
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
from xgboost import XGBClassifier

def train_model(combined_df, model_name='model',
                n_splits=5, gap=0,
                test_fraction=0.2):

    available = [c for c in FEATURE_COLS if c in combined_df.columns]
    X = combined_df[available]
    y = combined_df['label']

    tscv = TimeSeriesSplit(n_splits=n_splits, gap=gap)
    fold_results = []
    all_cms = []
```

```
    print(f'{model_name} running {n_splits}-fold TimeSeriesSplit CV\
n')

    for fold, (train_idx, val_idx) in enumerate(tscv.split(X)):
        X_tr_raw, X_val_raw = X.iloc[train_idx], X.iloc[val_idx]
        y_tr,     y_val     = y.iloc[train_idx], y.iloc[val_idx]

        sc = StandardScaler()
        X_tr  = sc.fit_transform(X_tr_raw)
        X_val = sc.transform(X_val_raw)

        w = compute_sample_weight('balanced', y_tr)

        m = XGBClassifier(
            n_estimators=500, max_depth=4, learning_rate=0.03,
            subsample=0.8, colsample_bytree=0.8,
            min_child_weight=10, gamma=0.1,
            reg_alpha=0.1, reg_lambda=1.0,
            eval_metric='mlogloss',
            early_stopping_rounds=30,
            random_state=42, n_jobs=-1,
        )
        m.fit(X_tr, y_tr, sample_weight=w,
              eval_set=[(X_val, y_val)], verbose=False)

        preds  = m.predict(X_val)

        # Save matrix for this fold
        cm = confusion_matrix(y_val, preds)
        all_cms.append(cm)

        report = classification_report(y_val, preds,

target_names=['SELL','HOLD','BUY'],
                                        output_dict=True,
                                        zero_division=0)

        fold_results.append({
            'fold'          : fold + 1,
            'buy_precision' : report['BUY']['precision'],
            'buy_recall'    : report['BUY']['recall'],
            'sell_precision': report['SELL']['precision'],
            'sell_recall'   : report['SELL']['recall'],
            'accuracy'      : report['accuracy'],
        })

        # FULL PRINT LOGIC RESTORED
        print(f"  (Fold {fold+1}) "
              f"BUY prec: {report['BUY']['precision']:.3f}, rec:
```

```
{report['BUY']['recall']:.3f} | "
              f"SELL prec: {report['SELL']['precision']:.3f}, rec:
{report['SELL']['recall']:.3f} | "
              f"Acc: {report['accuracy']:.3f}")

    #plotting
    print(f"\nPlotting fold results")
    # We add +1 to n_splits to make room for the "Total" matrix at the
end
    fig, axes = plt.subplots(1, n_splits + 1, figsize=(4 * (n_splits +
1), 4))

    # Plot individual folds
    for i, cm in enumerate(all_cms):
        disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=['SELL','HOLD','BUY'])
        disp.plot(cmap='Blues', ax=axes[i], colorbar=False)
        axes[i].set_title(f"Fold {i+1}")

    # Plot Total Aggregate Matrix
    total_cm = sum(all_cms)
    disp_total = ConfusionMatrixDisplay(confusion_matrix=total_cm,
display_labels=['SELL','HOLD','BUY'])
    disp_total.plot(cmap='Greens', ax=axes[-1], colorbar=False)
    axes[-1].set_title("TOTAL AGGREGATE")

    plt.tight_layout()
    plt.show()


    cv_df = pd.DataFrame(fold_results)
    print(f'\n[{model_name}] CV Averages')
    print(cv_df.mean(numeric_only=True).round(3).to_string())

    print(f'\nTraining final {model_name} on full dataset')
    final_scaler = StandardScaler()
    X_all = final_scaler.fit_transform(X)
    final_w = compute_sample_weight('balanced', y)

    final_model = XGBClassifier(
        n_estimators=500, max_depth=4, learning_rate=0.03,
        subsample=0.8, colsample_bytree=0.8,
        min_child_weight=10, gamma=0.1,
        reg_alpha=0.1, reg_lambda=1.0,
        eval_metric='mlogloss',
        random_state=42, n_jobs=-1,
    )
    final_model.fit(X_all, y, sample_weight=final_w, verbose=False)

    return final_model, final_scaler, available, cv_df
```

```
model, scaler, feature_cols, cv_results = train_model(combined,
model_name=MODEL_NAME)

models_registry = getattr(globals(), '__models__', {})
models_registry[MODEL_NAME] = {
    'model'   : model,
    'scaler'  : scaler,
    'features': feature_cols,
    'cv'      : cv_results,
}
```

```
xgboost running 5-fold TimeSeriesSplit CV

  (Fold 1) BUY prec: 0.306, rec: 0.362 | SELL prec: 0.206, rec: 0.252
| Acc: 0.444
  (Fold 2) BUY prec: 0.312, rec: 0.285 | SELL prec: 0.158, rec: 0.198
| Acc: 0.486
  (Fold 3) BUY prec: 0.398, rec: 0.531 | SELL prec: 0.213, rec: 0.187
| Acc: 0.424
  (Fold 4) BUY prec: 0.351, rec: 0.624 | SELL prec: 0.278, rec: 0.196
| Acc: 0.384
  (Fold 5) BUY prec: 0.351, rec: 0.355 | SELL prec: 0.209, rec: 0.270
| Acc: 0.413

Plotting fold results
```



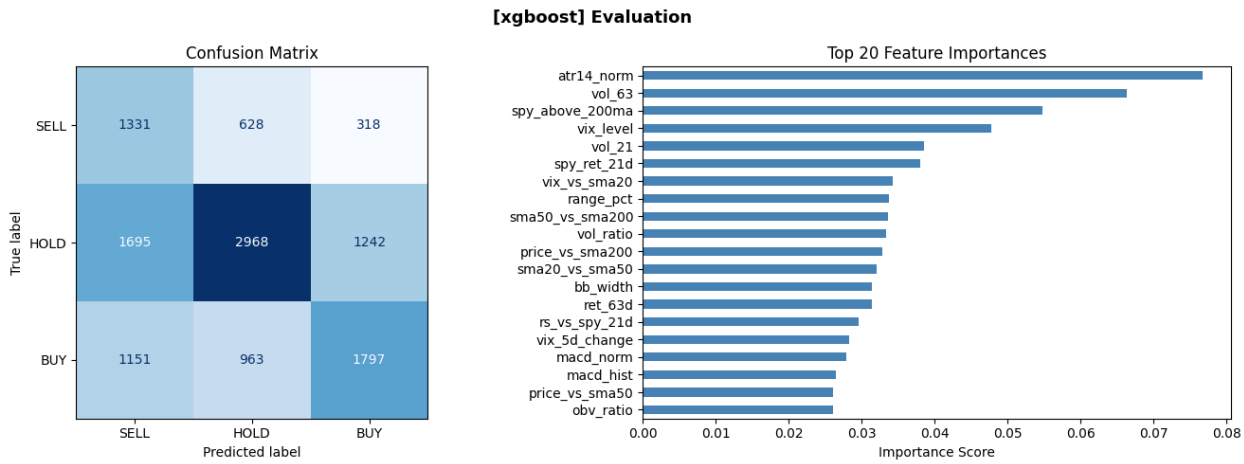

```
[xgboost] CV Averages
fold               3.000
buy_precision      0.344
buy_recall         0.431
sell_precision     0.213
sell_recall        0.221
accuracy           0.430

Training final xgboost on full dataset
```

```
from IPython.display import display, HTML

def evaluate_model(model, scaler, feature_cols, combined_df,
                   model_name='model', test_fraction=0.2):
    available = [c for c in feature_cols if c in combined_df.columns]
    X = combined_df[available]
```

```
    y = combined_df['label']

    split     = int(len(X) * (1 - test_fraction))
    X_tr_raw  = X.iloc[:split]
    X_te_raw  = X.iloc[split:]
    y_test    = y.iloc[split:]

    sc = StandardScaler()
    sc.fit(X_tr_raw)
    X_test = sc.transform(X_te_raw)
    preds  = model.predict(X_test)

    print(f'\n[{model_name}] Holdout Evaluation (last
{test_fraction*100:.0f}% of combined data)\n')
    print(classification_report(y_test, preds,
                                target_names=['SELL','HOLD','BUY']))

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    fig.suptitle(f'[{model_name}] Evaluation', fontsize=13,
fontweight='bold')

    # Confusion matrix
    cm = confusion_matrix(y_test, preds)
    ConfusionMatrixDisplay(cm,
display_labels=['SELL','HOLD','BUY']).plot(
        ax=axes[0], colorbar=False, cmap='Blues')
    axes[0].set_title('Confusion Matrix')

    # Feature importance
    imp = pd.Series(model.feature_importances_,
index=available).nlargest(20)
    imp.sort_values().plot(kind='barh', ax=axes[1], color='steelblue')
    axes[1].set_title('Top 20 Feature Importances')
    axes[1].set_xlabel('Importance Score')

    plt.tight_layout()
    plt.show()

    return preds

preds = evaluate_model(model, scaler, feature_cols, combined,
                       model_name=MODEL_NAME,
                       test_fraction=TEST_FRACTION)
```

```
[xgboost] Holdout Evaluation (last 20% of combined data)

              precision    recall  f1-score   support

        SELL       0.32      0.58      0.41      2277
        HOLD       0.65      0.50      0.57      5905
```

```
         BUY       0.54      0.46      0.49      3911

    accuracy                           0.50     12093
   macro avg       0.50      0.52      0.49     12093
weighted avg       0.55      0.50      0.51     12093
```



#Optuna Hyperparameter Tuning

Optuna intelligently searches hyperparameter combinations (Bayesian optimisation) and finds the best XGBoost config automatically.

```
OPTUNA_MODEL_NAME = 'xgboost_tuned'
OPTUNA_N_TRIALS   = 50
OPTUNA_CV_SPLITS  = 3
OPTUNA_CV_GAP     = 21
OPTUNA_METRIC     = 'accuracy'
OPTUNA_FRACTION   = TEST_FRACTION

def tune_xgboost(combined_df, model_name=OPTUNA_MODEL_NAME,
                 n_trials=OPTUNA_N_TRIALS, cv_splits=OPTUNA_CV_SPLITS,
                 cv_gap=OPTUNA_CV_GAP, metric=OPTUNA_METRIC,
                 test_fraction=OPTUNA_FRACTION):

    available = [c for c in FEATURE_COLS if c in combined_df.columns]
    X = combined_df[available]
    y = combined_df['label']

    split  = int(len(X) * (1 - test_fraction))
    X_tune = X.iloc[:split]
    y_tune = y.iloc[:split]
    tscv   = TimeSeriesSplit(n_splits=cv_splits, gap=cv_gap)

    def objective(trial):
        params = {
```

```
            'n_estimators'     : trial.suggest_int('n_estimators',
100, 800),
            'max_depth'        : trial.suggest_int('max_depth',
2,   8),
            'learning_rate'    : trial.suggest_float('learning_rate',
0.005, 0.3,  log=True),
            'subsample'        : trial.suggest_float('subsample',
0.5, 1.0),
            'colsample_bytree' :
trial.suggest_float('colsample_bytree', 0.4, 1.0),
            'min_child_weight' : trial.suggest_int('min_child_weight',
1,  30),
            'gamma'            : trial.suggest_float('gamma',
0.0, 1.0),
            'reg_alpha'        : trial.suggest_float('reg_alpha',
0.0, 2.0),
            'reg_lambda'       : trial.suggest_float('reg_lambda',
0.5, 5.0),
        }
        fold_scores = []
        for tr_idx, val_idx in tscv.split(X_tune):
            X_tr, X_val = X_tune.iloc[tr_idx], X_tune.iloc[val_idx]
            y_tr, y_val = y_tune.iloc[tr_idx], y_tune.iloc[val_idx]
            sc = StandardScaler()
            X_tr_s  = sc.fit_transform(X_tr)
            X_val_s = sc.transform(X_val)
            w = compute_sample_weight('balanced', y_tr)
            m = XGBClassifier(**params, eval_metric='mlogloss',
random_state=42, n_jobs=-1)
            m.fit(X_tr_s, y_tr, sample_weight=w, verbose=False)
            preds  = m.predict(X_val_s)
            report = classification_report(y_val, preds,

target_names=['SELL','HOLD','BUY'],
                                           output_dict=True,
zero_division=0)
            if metric == 'buy_f1':
fold_scores.append(report['BUY']['f1-score'])
            elif metric == 'buy_precision':
fold_scores.append(report['BUY']['precision'])
            else:
fold_scores.append(report['accuracy'])
        return np.mean(fold_scores)

    print(f'[{model_name}] Optuna: {n_trials} trials  metric={metric}
cv={cv_splits}-fold\n')
    study = optuna.create_study(direction='maximize')
    study.optimize(objective, n_trials=n_trials,
show_progress_bar=True)
```

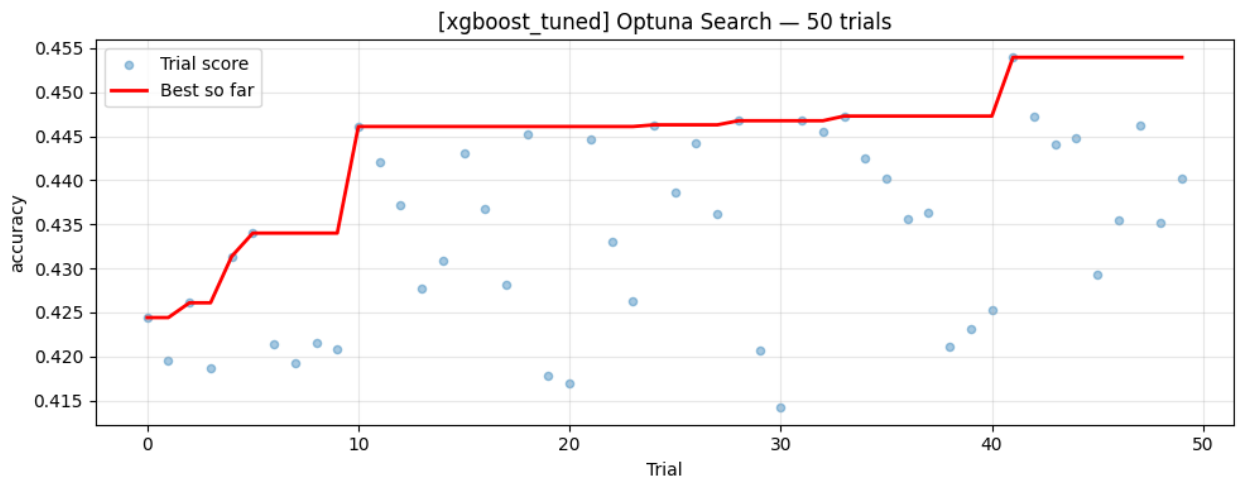
```
    best = study.best_params
    print(f'\n Best {metric}: {study.best_value:.4f}')
    print('  Best params:')
    for k, v in best.items():
        print(f'    {k:<22} {v}')

    # Train final model on ALL training data with best params
    print(f'\nTraining final [{model_name}] on full train set...')
    final_sc  = StandardScaler()
    X_all_s   = final_sc.fit_transform(X)
    w_all     = compute_sample_weight('balanced', y)
    best_model = XGBClassifier(**best, eval_metric='mlogloss',
random_state=42, n_jobs=-1)
    best_model.fit(X_all_s, y, sample_weight=w_all, verbose=False)
    print(f' [{model_name}] done')

    # Plot search history
    vals       = [t.value for t in study.trials]
    best_curve = [max(vals[:i+1]) for i in range(len(vals))]
    plt.figure(figsize=(10, 4))
    plt.scatter(range(len(vals)), vals, alpha=0.4, s=20, label='Trial
score')
    plt.plot(range(len(vals)), best_curve, color='red', linewidth=2,
label='Best so far')
    plt.xlabel('Trial')
    plt.ylabel(metric)
    plt.title(f'[{model_name}] Optuna Search — {n_trials} trials')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

    return best_model, final_sc, available, study


tuned_model, tuned_scaler, tuned_features, optuna_study =
tune_xgboost(combined)
```

```
[xgboost_tuned] Optuna: 50 trials  metric=accuracy  cv=3-fold
```

{"model_id":"55e05c2da9a347a990c37ec17966f143","version_major":2,"version_minor":0}

```
 Best accuracy: 0.4540
  Best params:
    n_estimators           623
    max_depth              8
    learning_rate          0.10855400587457993
    subsample              0.950941928222175
```

```
    colsample_bytree        0.42124513973765715
    min_child_weight        3
    gamma                   0.034808116951849245
    reg_alpha               0.7102748086858434
    reg_lambda              3.9668058064705973

Training final [xgboost_tuned] on full train set...
 [xgboost_tuned] done
```



#Threshold Tuning By default the model signals BUY/SELL whenever that class has the highest probability.
Threshold tuning adds a **minimum confidence requirement** before firing a signal.

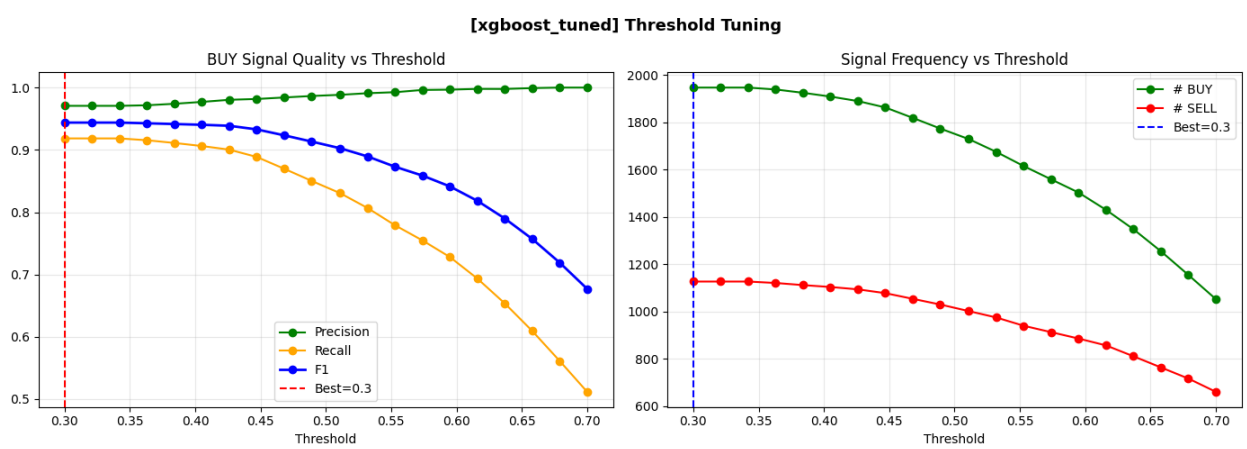
```
THRESH_MODEL_NAME = OPTUNA_MODEL_NAME
THRESH_FRACTION   = TEST_FRACTION
THRESH_MIN        = 0.30
THRESH_MAX        = 0.70
THRESH_STEPS      = 20

def tune_threshold(model, scaler, feature_cols, combined_df,
                   model_name='model', test_fraction=0.2,
                   thresh_min=0.30, thresh_max=0.70, steps=20):

    available = [c for c in feature_cols if c in combined_df.columns]
    X = combined_df[available]
    y = combined_df['label']

    # 80% train | 80-90% = tune | 90-100% = untouched holdout
    train_end = int(len(X) * 0.80)
    val_end   = int(len(X) * 0.90)
    sc = StandardScaler()
    sc.fit(X.iloc[:train_end])
    X_val_s = sc.transform(X.iloc[train_end:val_end])
    y_val   = y.iloc[train_end:val_end]
```

```
    proba   = model.predict_proba(X_val_s)

    records = []
    for t in np.linspace(thresh_min, thresh_max, steps):
        preds = [int(np.argmax(p)) if p.max() >= t else 1 for p in
proba]
        preds = np.array(preds)
        rep   = classification_report(y_val, preds,

target_names=['SELL','HOLD','BUY'],
                                          output_dict=True,
zero_division=0)
        records.append({
            'threshold'     : round(t, 3),
            'buy_precision' : round(rep['BUY']['precision'], 4),
            'buy_recall'    : round(rep['BUY']['recall'],    4),
            'buy_f1'        : round(rep['BUY']['f1-score'],  4),
            'sell_f1'       : round(rep['SELL']['f1-score'], 4),
            'accuracy'      : round(rep['accuracy'],         4),
            'n_buy'         : int((preds == 2).sum()),
            'n_sell'        : int((preds == 0).sum()),
        })

    results_df  = pd.DataFrame(records)
    best_idx    = results_df['buy_f1'].idxmax()
    best_thresh = results_df.loc[best_idx, 'threshold']

    print(f'[{model_name}] Threshold Search')
    print(results_df.to_string(index=False))
    print(f'\n  Best threshold : {best_thresh}')
    print(f'     BUY F1        : {results_df.loc[best_idx,
"buy_f1"]:.4f}')
    print(f'     BUY Precision : {results_df.loc[best_idx,
"buy_precision"]:.4f}')
    print(f'     BUY signals   : {results_df.loc[best_idx, "n_buy"]}')

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    fig.suptitle(f'[{model_name}] Threshold Tuning', fontsize=13,
fontweight='bold')
    axes[0].plot(results_df['threshold'], results_df['buy_precision'],
marker='o', label='Precision', color='green')
    axes[0].plot(results_df['threshold'], results_df['buy_recall'],
marker='o', label='Recall',    color='orange')
    axes[0].plot(results_df['threshold'], results_df['buy_f1'],
marker='o', label='F1',        color='blue', linewidth=2)
    axes[0].axvline(best_thresh, color='red', linestyle='--',
linewidth=1.5, label=f'Best={best_thresh}')
    axes[0].set_xlabel('Threshold')
    axes[0].set_title('BUY Signal Quality vs Threshold')
    axes[0].legend()
```

```
    axes[0].grid(True, alpha=0.3)
    axes[1].plot(results_df['threshold'], results_df['n_buy'],
marker='o', color='green', label='# BUY')
    axes[1].plot(results_df['threshold'], results_df['n_sell'],
marker='o', color='red',   label='# SELL')
    axes[1].axvline(best_thresh, color='blue', linestyle='--',
linewidth=1.5, label=f'Best={best_thresh}')
    axes[1].set_xlabel('Threshold')
    axes[1].set_title('Signal Frequency vs Threshold')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

    return best_thresh, results_df


best_threshold, threshold_results = tune_threshold(
    tuned_model, tuned_scaler, tuned_features, combined,
    model_name    = THRESH_MODEL_NAME,
    test_fraction = THRESH_FRACTION,
    thresh_min    = THRESH_MIN,
    thresh_max    = THRESH_MAX,
    steps         = THRESH_STEPS,
)
```

```
[xgboost_tuned] Threshold Search
 threshold  buy_precision  buy_recall  buy_f1  sell_f1  accuracy
n_buy  n_sell
     0.300         0.9707      0.9184  0.9438   0.9522    0.9494
1947    1127
     0.321         0.9707      0.9184  0.9438   0.9522    0.9494
1947    1127
     0.342         0.9707      0.9184  0.9438   0.9522    0.9494
1947    1127
     0.363         0.9716      0.9155  0.9427   0.9529    0.9486
1939    1121
     0.384         0.9740      0.9111  0.9415   0.9532    0.9467
1925    1112
     0.405         0.9770      0.9062  0.9403   0.9521    0.9451
1909    1104
     0.426         0.9804      0.9004  0.9387   0.9492    0.9429
1890    1094
     0.447         0.9817      0.8887  0.9329   0.9452    0.9375
1863    1078
     0.468         0.9841      0.8698  0.9234   0.9445    0.9312
1819    1054
     0.489         0.9865      0.8503  0.9134   0.9421    0.9246
1774    1030
     0.511         0.9884      0.8304  0.9026   0.9375    0.9171
```

```
1729    1002
     0.532          0.9910       0.8066  0.8894   0.9276    0.9067
1675     975
     0.553          0.9926       0.7789  0.8729   0.9112    0.8927
1615     940
     0.574          0.9962       0.7546  0.8587   0.8987    0.8816
1559     913
     0.595          0.9967       0.7279  0.8413   0.8848    0.8685
1503     886
     0.616          0.9979       0.6934  0.8182   0.8693    0.8526
1430     857
     0.637          0.9978       0.6535  0.7898   0.8433    0.8318
1348     811
     0.658          0.9992       0.6093  0.7570   0.8154    0.8096
1255     764
     0.679          1.0000       0.5612  0.7190   0.7849    0.7856
1155     717
     0.700          1.0000       0.5117  0.6770   0.7465    0.7595
1053     661

  Best threshold : 0.3
     BUY F1        : 0.9438
     BUY Precision : 0.9707
     BUY signals   : 1947
```




```python
def predict_with_threshold(proba_array, threshold):
    return np.array([int(np.argmax(p)) if p.max() >= threshold else 1
                     for p in proba_array])


# Demo: tuned model + best threshold on holdout
available = [c for c in tuned_features if c in combined.columns]
X_all     = combined[available]
split     = int(len(X_all) * (1 - TEST_FRACTION))
sc_demo   = StandardScaler()
```

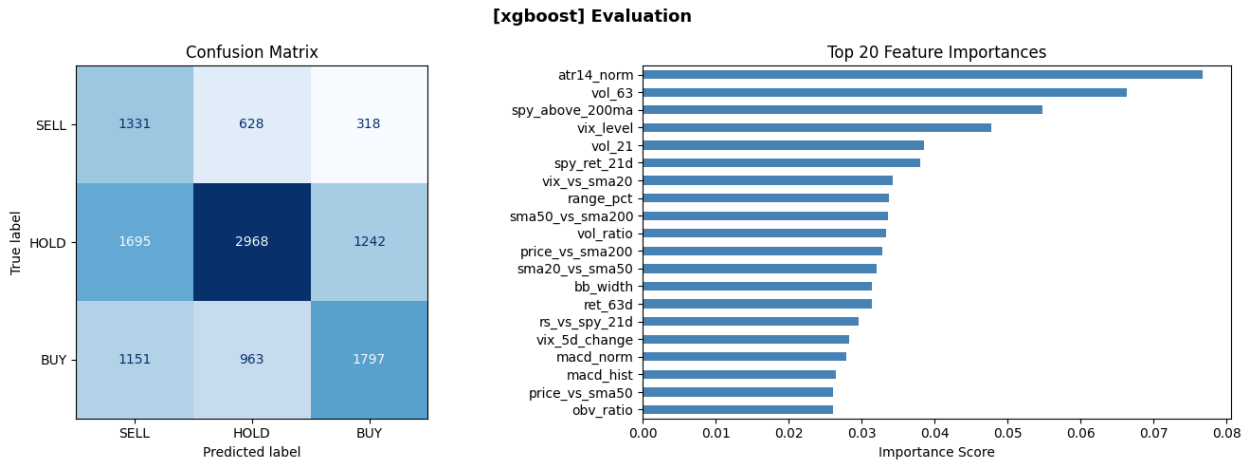
```
sc_demo.fit(X_all.iloc[:split])
X_test_s  = sc_demo.transform(X_all.iloc[split:])
y_test    = combined['label'].iloc[split:]

proba_test   = tuned_model.predict_proba(X_test_s)
preds_thresh = predict_with_threshold(proba_test, best_threshold)

print(f'[{THRESH_MODEL_NAME}] Holdout results with threshold =
{best_threshold}')
print(classification_report(y_test, preds_thresh,
                            target_names=['SELL','HOLD','BUY'],
zero_division=0))
print(f'Signals BUY: {(preds_thresh==2).sum()}  '
      f'SELL: {(preds_thresh==0).sum()}  '
      f'HOLD: {(preds_thresh==1).sum()}')
```

```
[xgboost_tuned] Holdout results with threshold = 0.3
              precision    recall  f1-score   support

        SELL       0.94      0.97      0.95      2277
        HOLD       0.95      0.97      0.96      5905
         BUY       0.97      0.93      0.95      3911

    accuracy                           0.95     12093
   macro avg       0.95      0.95      0.95     12093
weighted avg       0.95      0.95      0.95     12093

Signals BUY: 3741  SELL: 2339  HOLD: 6013
```

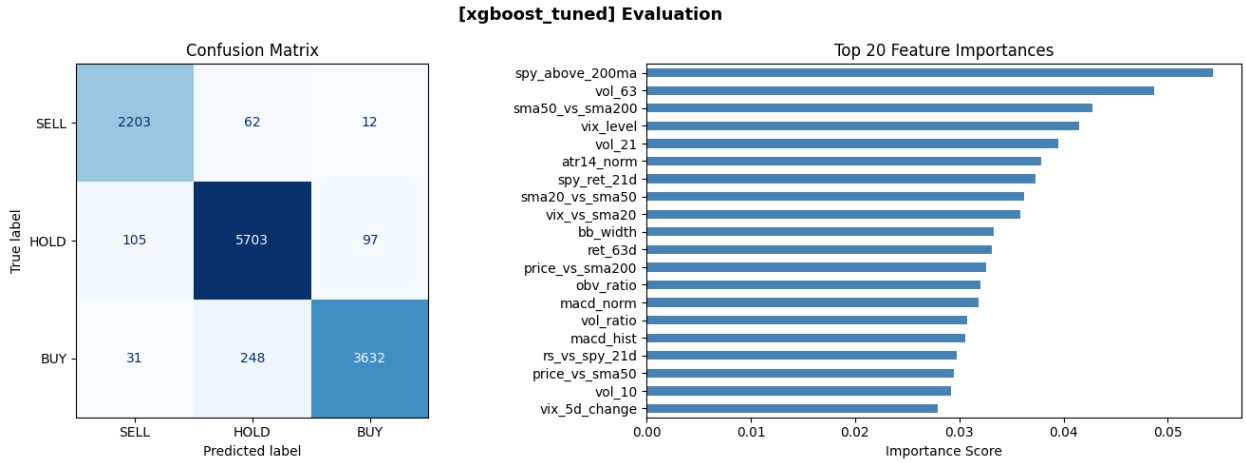
```
print('Baseline XGBoost ')
_ = evaluate_model(model, scaler, feature_cols, combined,
                   model_name=MODEL_NAME, test_fraction=TEST_FRACTION)
print('\nTuned XGBoost')
_ = evaluate_model(tuned_model, tuned_scaler, tuned_features,
combined,
                   model_name=OPTUNA_MODEL_NAME,
test_fraction=TEST_FRACTION)
```

```
Baseline XGBoost

[xgboost] Holdout Evaluation (last 20% of combined data)

              precision    recall  f1-score   support

        SELL       0.32      0.58      0.41      2277
        HOLD       0.65      0.50      0.57      5905
         BUY       0.54      0.46      0.49      3911

    accuracy                           0.50     12093
   macro avg       0.50      0.52      0.49     12093
```

```
weighted avg       0.55      0.50      0.51     12093
```




```
Tuned XGBoost

[xgboost_tuned] Holdout Evaluation (last 20% of combined data)

              precision    recall  f1-score   support

        SELL       0.94      0.97      0.95      2277
        HOLD       0.95      0.97      0.96      5905
         BUY       0.97      0.93      0.95      3911

    accuracy                           0.95     12093
   macro avg       0.95      0.95      0.95     12093
weighted avg       0.95      0.95      0.95     12093
```



#Backtesting

```
BT_MODEL_NAME  = OPTUNA_MODEL_NAME
BT_TICKER      = 'AMZN'
BT_START       = None
BT_END         = None
BT_CASH        = 10_000
BT_COMMISSION  = 0.003    # 0.3% per trade
BT_FRACTION    = TEST_FRACTION  # must match what was used in training

def run_backtest(model, scaler, feature_cols, combined_df, stock_data,
                 ticker=BT_TICKER, test_fraction=BT_FRACTION,
                 bt_start=BT_START, bt_end=BT_END,
                 cash=BT_CASH, commission=BT_COMMISSION,
                 model_name=BT_MODEL_NAME):

    df_t = combined_df[combined_df['ticker'] == ticker].copy()
    if len(df_t) == 0:
        print(f"'{ticker}' not found. Available:
{list(combined_df['ticker'].unique())}")
        return None, None

    available = [c for c in feature_cols if c in df_t.columns]
    X = df_t[available]
    split = int(len(X) * (1 - test_fraction))

    sc = StandardScaler()
    sc.fit(X.iloc[:split])
    X_test_scaled = sc.transform(X.iloc[split:])
    signals = model.predict(X_test_scaled)

    # Build OHLCV from raw stock_data (combined dropped these columns)
    raw = stock_data[ticker].copy()
    raw.columns = [c[0] if isinstance(c, tuple) else c for c in
raw.columns]
    bt_df = raw.loc[X.iloc[split:].index,
                    ['Open','High','Low','Close','Volume']].copy()
    bt_df.index     = pd.to_datetime(bt_df.index)
    bt_df['signal'] = signals

    if bt_start:
        bt_df = bt_df[bt_df.index >= pd.to_datetime(bt_start)]
    if bt_end:
        bt_df = bt_df[bt_df.index <= pd.to_datetime(bt_end)]
    if len(bt_df) < 30:
        print(f'Too few rows after date filter ({len(bt_df)}). Widen
the range.')
        return None, None

    class MLStrategy(Strategy):
        def init(self):
            self.ml_signal = self.I(
```

```
                lambda: self.data.df['signal'].values,
                name='ML Signal'
            )
        def next(self):
            sig = self.ml_signal[-1]
            if   sig == 2 and not self.position: self.buy()
            elif sig == 0 and     self.position: self.position.close()

    bt    = Backtest(bt_df, MLStrategy, cash=cash,
                     commission=commission, exclusive_orders=True)
    stats = bt.run()

    date_range = f"from {str(stats['Start'])[:10]} to
{str(stats['End'])[:10]}"
    print(f"\n{'='*60}")
    print(f"  BACKTESTING — {ticker} {date_range}")
    print(f"{'='*60}")
    rows = [
        ('Starting cash',     f"${cash:,.0f}"),
        ('Final equity',      f"${stats['Equity Final [$]']:,.2f}"),
        ('Strategy return',   f"{stats['Return [%]']:.2f}%"),
        ('Buy & Hold return', f"{stats['Buy & Hold Return [%]']:.2f}
%"),
        ('Max drawdown',      f"{stats['Max. Drawdown [%]']:.2f}%"),
        ('Sharpe ratio',      f"{stats['Sharpe Ratio']:.3f}"),
        ('Win rate',          f"{stats['Win Rate [%]']:.1f}%"),
        ('No. of Trades',          stats['# Trades']),
        ('Avg trade return',  f"{stats['Avg. Trade [%]']:.2f}%"),
        ('Profit factor',     f"{stats['Profit Factor']:.2f}" if
stats['Profit Factor'] else 'N/A'),
        ('Calmar ratio',      f"{stats['Calmar Ratio']:.3f}"),
    ]
    for name, val in rows:
        print(f'  {name:<24} {val}')
    print(f"\n  Beat Buy&Hold: {'Yes' if stats['Return [%]'] >
stats['Buy & Hold Return [%]'] else 'No'}")
    print(f"  Sharpe > 1.0:  {'Yes' if stats['Sharpe Ratio'] > 1 else
'No'}")

    return stats, bt


stats, bt = run_backtest(tuned_model, tuned_scaler, tuned_features,
combined, stock_data)
```

{"model_id":"de2fd9fc98534376892c7e5918d813f9","version_major":2,"version_minor":0}

```
============================================================
```

```
  BACKTESTING — AMZN from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $56,469.05
  Strategy return          464.69%
  Buy & Hold return        156.80%
  Max drawdown             -12.43%
  Sharpe ratio             1.646
  Win rate                 100.0%
  No. of Trades            15
  Avg trade return         11.80%
  Profit factor            nan
  Calmar ratio             6.186

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

```
all_stats = {}
total_invested = 0

for ticker in stock_data.keys():
    stats, bt = run_backtest(
        tuned_model, tuned_scaler, tuned_features, combined,
stock_data,
        ticker=ticker
    )
    if stats is not None:
        all_stats[ticker] = stats
        total_invested += BT_CASH

# Summary
total_final   = sum(s['Equity Final [$]'] for s in all_stats.values())
total_profit  = total_final - total_invested
total_return  = (total_profit / total_invested) * 100 if
total_invested > 0 else 0

summary_text = (
    f"\n{'='*40}\n"
    f" XGBOOST TUNED OVERALL PORTFOLIO SUMMARY\n"
    f"{'='*40}\n"
    f"  Tickers traded       {len(all_stats)}\n"
    f"  Total invested       ${total_invested:,.0f}\n"
    f"  Total final equity   ${total_final:,.2f}\n"
    f"  Total profit/loss    ${total_profit:,.2f}\n"
    f"  Overall return       {total_return:.2f}%\n"
)

tuned_xgboost = {
    "tickers_traded": len(all_stats),
    "total_invested": total_invested,
```

```
    "total_final_equity": total_final,
    "total_profit_loss": total_profit,
    "overall_return_percent": total_return,
    "per_ticker_stats": all_stats,
    "summary": summary_text
}

print(tuned_xgboost["summary"])
```

{"model_id":"07da10b833824c6fbb4798d9b6c6df69","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — AAPL from 2022-12-12 to 2025-12-30
============================================================
  Starting cash            $10,000
  Final equity             $19,777.38
  Strategy return          97.77%
  Buy & Hold return        91.85%
  Max drawdown             -14.39%
  Sharpe ratio             1.040
  Win rate                 86.7%
  No. of Trades            15
  Avg trade return         4.09%
  Profit factor            7.39
  Calmar ratio             1.751

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"f7c3d884fb394d8ca32e4095bb9fe9d3","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — MSFT from 2022-12-12 to 2025-12-30
============================================================
  Starting cash            $10,000
  Final equity             $19,091.54
  Strategy return          90.92%
  Buy & Hold return        97.67%
  Max drawdown             -15.24%
  Sharpe ratio             1.056
  Win rate                 69.2%
  No. of Trades            13
  Avg trade return         5.14%
  Profit factor            5.63
  Calmar ratio             1.557
```

```
  Beat Buy&Hold: No
  Sharpe > 1.0:  Yes
```

{"model_id":"c83ac84b492a405d83c573d6c3f8dc77","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — GOOGL from 2022-12-12 to 2025-12-30
============================================================
  Starting cash            $10,000
  Final equity             $24,392.94
  Strategy return          143.93%
  Buy & Hold return        238.89%
  Max drawdown             -20.26%
  Sharpe ratio             1.071
  Win rate                 55.9%
  No. of Trades            34
  Avg trade return         2.68%
  Profit factor            2.90
  Calmar ratio             1.686

  Beat Buy&Hold: No
  Sharpe > 1.0:  Yes
```

{"model_id":"ef8ec21c19aa4585aca64f50201d5ff3","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — AMZN from 2022-12-12 to 2025-12-30
============================================================
  Starting cash            $10,000
  Final equity             $56,469.05
  Strategy return          464.69%
  Buy & Hold return        156.80%
  Max drawdown             -12.43%
  Sharpe ratio             1.646
  Win rate                 100.0%
  No. of Trades            15
  Avg trade return         11.80%
  Profit factor            nan
  Calmar ratio             6.186

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"70ec9139904e4b6497eeb56128018435","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — META from 2023-06-06 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $21,647.13
  Strategy return          116.47%
  Buy & Hold return        147.35%
  Max drawdown             -20.32%
  Sharpe ratio             0.989
  Win rate                 61.5%
  No. of Trades            13
  Avg trade return         5.52%
  Profit factor            7.64
  Calmar ratio             1.733

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"2395227e5b744c619d32b11d4f6fa4b7","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — JPM from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $36,432.79
  Strategy return          264.33%
  Buy & Hold return        159.47%
  Max drawdown             -5.88%
  Sharpe ratio             2.101
  Win rate                 83.3%
  No. of Trades            12
  Avg trade return         10.85%
  Profit factor            52.56
  Calmar ratio             9.032

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"16a25a2cbeeb4c648235aebbee900c50","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — BAC from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $25,082.73
  Strategy return          150.83%
```

```
  Buy & Hold return         82.72%
  Max drawdown              -8.18%
  Sharpe ratio              1.653
  Win rate                  72.7%
  No. of Trades             11
  Avg trade return          7.58%
  Profit factor             11.64
  Calmar ratio              4.323

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"ea7b8888c9c644efbd2e17c071dc26b3","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — GS from 2022-12-12 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $32,216.54
  Strategy return           222.17%
  Buy & Hold return         162.58%
  Max drawdown              -11.90%
  Sharpe ratio              1.560
  Win rate                  63.6%
  No. of Trades             11
  Avg trade return          9.80%
  Profit factor             18.79
  Calmar ratio              3.951

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"047cfe26f86843cdb7384ed9fde7605f","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — XOM from 2022-12-12 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $14,899.88
  Strategy return           49.00%
  Buy & Hold return         26.44%
  Max drawdown              -13.56%
  Sharpe ratio              0.804
  Win rate                  68.2%
  No. of Trades             22
  Avg trade return          1.76%
  Profit factor             3.71
```

```
  Calmar ratio              1.035

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"80efc10be11f4e19aef2873df10ee6c8","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — CVX from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $17,114.79
  Strategy return           71.15%
  Buy & Hold return         1.78%
  Max drawdown              -11.29%
  Sharpe ratio              1.204
  Win rate                  66.7%
  No. of Trades             18
  Avg trade return          3.05%
  Profit factor             7.28
  Calmar ratio              1.715

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"44566f1bce174846bf1ceb07a0435922","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — JNJ from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $8,856.33
  Strategy return           -11.44%
  Buy & Hold return         27.46%
  Max drawdown              -23.16%
  Sharpe ratio              -0.403
  Win rate                  31.8%
  No. of Trades             22
  Avg trade return          -0.56%
  Profit factor             0.66
  Calmar ratio              -0.169

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"8a1dfe2416b047aa8b8788e287d8d530","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — PFE from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $10,892.54
  Strategy return          8.93%
  Buy & Hold return        -43.01%
  Max drawdown             -30.42%
  Sharpe ratio             0.160
  Win rate                 59.1%
  No. of Trades            22
  Avg trade return         0.39%
  Profit factor            1.30
  Calmar ratio             0.094

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"9db83385d7d74f31ad9be12672ce17b8","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — WMT from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $17,395.64
  Strategy return          73.96%
  Buy & Hold return        135.29%
  Max drawdown             -24.74%
  Sharpe ratio             0.976
  Win rate                 57.7%
  No. of Trades            26
  Avg trade return         1.79%
  Profit factor            3.36
  Calmar ratio             0.809

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"b0c921ab4f1d45a4a8dc70f7d3191f95","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — HD from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $12,854.31
  Strategy return          28.54%
```

```
  Buy & Hold return         13.97%
  Max drawdown              -17.94%
  Sharpe ratio              0.653
  Win rate                  68.4%
  No. of Trades             19
  Avg trade return          1.37%
  Profit factor             2.54
  Calmar ratio              0.481

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"5ae1c3391a2c40f39885a8459f933a21","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — TSLA from 2023-01-18 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $25,067.11
  Strategy return           150.67%
  Buy & Hold return         252.87%
  Max drawdown              -44.26%
  Sharpe ratio              0.540
  Win rate                  46.7%
  No. of Trades             30
  Avg trade return          3.12%
  Profit factor             2.06
  Calmar ratio              0.829

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"86c33137a49a40bdaeb27088015a722f","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — NVDA from 2022-12-12 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $43,676.05
  Strategy return           336.76%
  Buy & Hold return         970.59%
  Max drawdown              -27.41%
  Sharpe ratio              0.979
  Win rate                  70.7%
  No. of Trades             41
  Avg trade return          3.50%
  Profit factor             3.83
```

```
  Calmar ratio              2.281

  Beat Buy&Hold: No
  Sharpe > 1.0:  No

========================================
 XGBOOST TUNED OVERALL PORTFOLIO SUMMARY
========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $385,866.75
  Total profit/loss    $225,866.75
  Overall return       141.17%
```

```
all_stats = {}
total_invested = 0

for ticker in stock_data.keys():
    stats, bt = run_backtest(
        model, scaler, feature_cols, combined, stock_data,
        ticker=ticker
    )
    if stats is not None:
        all_stats[ticker] = stats
        total_invested += BT_CASH

# Summary
total_final   = sum(s['Equity Final [$]'] for s in all_stats.values())
total_profit  = total_final - total_invested
total_return  = (total_profit / total_invested) * 100 if
total_invested > 0 else 0

# Build formatted summary
summary_text = (
    f"\n{'='*40}\n"
    f"XGBoost  OVERALL PORTFOLIO SUMMARY\n"
    f"{'='*40}\n"
    f"  Tickers traded       {len(all_stats)}\n"
    f"  Total invested       ${total_invested:,.0f}\n"
    f"  Total final equity   ${total_final:,.2f}\n"
    f"  Total profit/loss    ${total_profit:,.2f}\n"
    f"  Overall return       {total_return:.2f}%\n"
)

result_xgboost = {
    "tickers_traded": len(all_stats),
    "total_invested": total_invested,
    "total_final_equity": total_final,
    "total_profit_loss": total_profit,
```

```
    "overall_return_percent": total_return,
    "per_ticker_stats": all_stats,
    "summary": summary_text
}

print(result_xgboost["summary"])
```

{"model_id":"173283202b77436cbe7a71a3a66d57c1","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — AAPL from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $11,950.14
  Strategy return          19.50%
  Buy & Hold return        91.85%
  Max drawdown             -17.62%
  Sharpe ratio             0.314
  Win rate                 56.2%
  No. of Trades            16
  Avg trade return         1.12%
  Profit factor            1.99
  Calmar ratio             0.343

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"a32acc6d311a40f0ae9e775d1756936b","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — MSFT from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $13,326.17
  Strategy return          33.26%
  Buy & Hold return        97.67%
  Max drawdown             -23.16%
  Sharpe ratio             0.615
  Win rate                 40.0%
  No. of Trades            15
  Avg trade return         1.99%
  Profit factor            3.00
  Calmar ratio             0.428

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"5d3bf549df38490f994abf78c893c4a6","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — GOOGL from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $15,475.86
  Strategy return          54.76%
  Buy & Hold return        238.89%
  Max drawdown             -20.44%
  Sharpe ratio             0.682
  Win rate                 54.1%
  No. of Trades            61
  Avg trade return         0.72%
  Profit factor            1.88
  Calmar ratio             0.757

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"ba08dacda6d9480d9317b3ec167f8536","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — AMZN from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $30,325.35
  Strategy return          203.25%
  Buy & Hold return        156.80%
  Max drawdown             -12.42%
  Sharpe ratio             1.377
  Win rate                 68.8%
  No. of Trades            32
  Avg trade return         3.55%
  Profit factor            9.47
  Calmar ratio             3.553

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"dcb94c7c04654c9aae0a04bf23a10f8f","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — META from 2023-06-06 to 2025-12-30
================================================================
```

```
  Starting cash             $10,000
  Final equity              $16,432.55
  Strategy return           64.33%
  Buy & Hold return         147.35%
  Max drawdown              -21.95%
  Sharpe ratio              0.815
  Win rate                  78.6%
  No. of Trades             14
  Avg trade return          3.65%
  Profit factor             4.08
  Calmar ratio              0.976

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"4332743f9ab94a78ad7c7b9e17e8de7d","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — JPM from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $18,788.83
  Strategy return           87.89%
  Buy & Hold return         159.47%
  Max drawdown              -12.43%
  Sharpe ratio              1.537
  Win rate                  76.9%
  No. of Trades             13
  Avg trade return          5.02%
  Profit factor             9.04
  Calmar ratio              1.857

  Beat Buy&Hold: No
  Sharpe > 1.0:  Yes
```

{"model_id":"71fcad8d8d9a44c9a4e13ce33a043eb2","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — BAC from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $9,656.67
  Strategy return           -3.43%
  Buy & Hold return         82.72%
  Max drawdown              -23.94%
  Sharpe ratio              -0.104
  Win rate                  38.5%
```

```
  No. of Trades             13
  Avg trade return          -0.27%
  Profit factor             0.94
  Calmar ratio              -0.048

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"71dbfa90daa0482c84eef3abce815e34","version_major":2,"version_minor":0}

```
==============================================================
  BACKTESTING — GS from 2022-12-12 to 2025-12-30
==============================================================
  Starting cash             $10,000
  Final equity              $15,139.03
  Strategy return           51.39%
  Buy & Hold return         162.58%
  Max drawdown              -15.45%
  Sharpe ratio              0.860
  Win rate                  68.4%
  No. of Trades             19
  Avg trade return          2.25%
  Profit factor             2.93
  Calmar ratio              0.948

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"38dfb8dc09d44c719a3620f33d7155e6","version_major":2,"version_minor":0}

```
==============================================================
  BACKTESTING — XOM from 2022-12-12 to 2025-12-30
==============================================================
  Starting cash             $10,000
  Final equity              $10,471.26
  Strategy return           4.71%
  Buy & Hold return         26.44%
  Max drawdown              -17.18%
  Sharpe ratio              0.117
  Win rate                  60.6%
  No. of Trades             33
  Avg trade return          0.14%
  Profit factor             1.17
  Calmar ratio              0.089

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"5ef09a682e5d4df4a1efda7348616e19","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — CVX from 2022-12-12 to 2025-12-30
============================================================
  Starting cash            $10,000
  Final equity             $11,647.45
  Strategy return          16.47%
  Buy & Hold return        1.78%
  Max drawdown             -15.38%
  Sharpe ratio             0.347
  Win rate                 54.2%
  No. of Trades            24
  Avg trade return         0.69%
  Profit factor            1.72
  Calmar ratio             0.335

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"476d646ce53d4f40a9c2b4b97910a41a","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — JNJ from 2022-12-12 to 2025-12-30
============================================================
  Starting cash            $10,000
  Final equity             $8,371.95
  Strategy return          -16.28%
  Buy & Hold return        27.46%
  Max drawdown             -22.72%
  Sharpe ratio             -0.736
  Win rate                 40.6%
  No. of Trades            32
  Avg trade return         -0.56%
  Profit factor            0.57
  Calmar ratio             -0.250

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"d83ae3ad96c94897a08382148a04f83c","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — PFE from 2022-12-12 to 2025-12-30
============================================================
```

```
  Starting cash             $10,000
  Final equity              $7,279.81
  Strategy return           -27.20%
  Buy & Hold return         -43.01%
  Max drawdown              -38.75%
  Sharpe ratio              -0.716
  Win rate                  44.8%
  No. of Trades             58
  Avg trade return          -0.55%
  Profit factor             0.68
  Calmar ratio              -0.256

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"5ab73dda755f489c8d763db9013bbc00","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — WMT from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $18,363.89
  Strategy return           83.64%
  Buy & Hold return         135.29%
  Max drawdown              -8.96%
  Sharpe ratio              1.297
  Win rate                  51.7%
  No. of Trades             29
  Avg trade return          2.13%
  Profit factor             4.07
  Calmar ratio              2.474

  Beat Buy&Hold: No
  Sharpe > 1.0:  Yes
```

{"model_id":"7e14337b38824640a99e96a6f336df00","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — HD from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $10,837.13
  Strategy return           8.37%
  Buy & Hold return         13.97%
  Max drawdown              -15.08%
  Sharpe ratio              0.243
  Win rate                  44.0%
```

```
  No. of Trades             25
  Avg trade return          0.33%
  Profit factor             1.40
  Calmar ratio              0.178

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"9ecb12d0506f40cb9bca636423d26e5c","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — TSLA from 2023-01-18 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $7,591.15
  Strategy return           -24.09%
  Buy & Hold return         252.87%
  Max drawdown              -57.82%
  Sharpe ratio              -0.207
  Win rate                  46.4%
  No. of Trades             56
  Avg trade return          -0.51%
  Profit factor             1.06
  Calmar ratio              -0.155

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"621e9d264e4045a2b5a16f8fcdf00fea","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — NVDA from 2022-12-12 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $10,390.23
  Strategy return           3.90%
  Buy & Hold return         970.59%
  Max drawdown              -49.02%
  Sharpe ratio              0.041
  Win rate                  49.1%
  No. of Trades             55
  Avg trade return          0.07%
  Profit factor             1.14
  Calmar ratio              0.026

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

```
========================================
XGBoost  OVERALL PORTFOLIO SUMMARY
========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $216,047.47
  Total profit/loss    $56,047.47
  Overall return       35.03%
```

```
print(tuned_xgboost["summary"])
print(result_xgboost["summary"])
```

```
========================================
 XGBOOST TUNED OVERALL PORTFOLIO SUMMARY
========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $385,866.75
  Total profit/loss    $225,866.75
  Overall return       141.17%
```

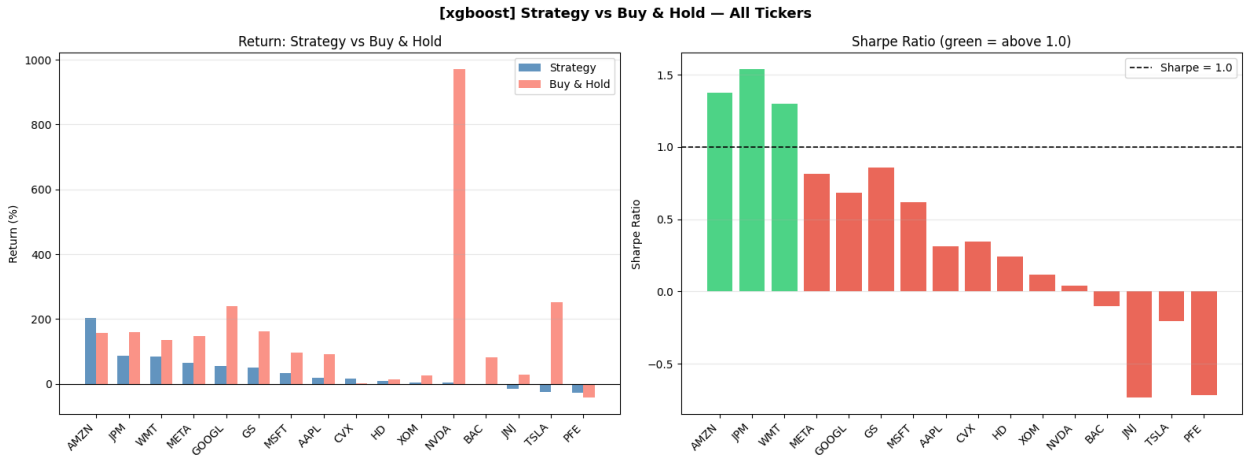
```
========================================
XGBoost  OVERALL PORTFOLIO SUMMARY
========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $216,047.47
  Total profit/loss    $56,047.47
  Overall return       35.03%
```

```
import pandas as pd
import matplotlib.pyplot as plt

def plot_backtest_summary(result_obj, model_name="model"):
    # Build summary DataFrame from the saved result
    summary_df = pd.DataFrame(result_obj["per_ticker_stats"]).T

    # Keep only the columns we want and rename them
    summary_df = summary_df[[
        'Return [%]',
        'Buy & Hold Return [%]',
        'Sharpe Ratio',
        'Max. Drawdown [%]',
        'Win Rate [%]',
        '# Trades',
        'Avg. Trade [%]',
```

```
        'Profit Factor'
    ]].copy()

    summary_df.columns = [
        'Return %',
        'Buy & Hold %',
        'Sharpe',
        'Max DD %',
        'Win Rate %',
        '# Trades',
        'Avg Trade %',
        'Profit Factor'
    ]

    summary_df['Beat B&H'] = (
        summary_df['Return %'] > summary_df['Buy & Hold %']
    ).map({True: 'Yes', False: 'No'})

    summary_df['Sharpe OK'] = (
        summary_df['Sharpe'] > 1.0
    ).map({True: 'Yes', False: 'No'})

    summary_df = summary_df.sort_values('Return %', ascending=False)

    # Print table
    divider = '=' * 72
    print('\n' + divider)
    print(f'  [{model_name}] ALL-TICKER BACKTEST SUMMARY')
    print(divider)
    print(summary_df.to_string())

    beat_bh = (summary_df['Beat B&H'] == 'Yes').sum()
    good_sharpe = (summary_df['Sharpe OK'] == 'Yes').sum()

    print(f'\n  Tickers beating Buy & Hold : {beat_bh} /
{len(summary_df)}')
    print(f'  Tickers with Sharpe > 1.0  : {good_sharpe} /
{len(summary_df)}')
    print(f'  Avg strategy return        : {summary_df["Return
%"].mean():.2f}%')
    print(f'  Avg buy & hold return      : {summary_df["Buy & Hold
%"].mean():.2f}%')

    # Plot
    fig, axes = plt.subplots(1, 2, figsize=(16, 6))
    fig.suptitle(f'[{model_name}] Strategy vs Buy & Hold — All
Tickers',
                 fontsize=13, fontweight='bold')

    x = range(len(summary_df))
```

```
    width = 0.35

    axes[0].bar(
        [i - width/2 for i in x],
        summary_df['Return %'],
        width,
        label='Strategy',
        color='steelblue',
        alpha=0.85
    )
    axes[0].bar(
        [i + width/2 for i in x],
        summary_df['Buy & Hold %'],
        width,
        label='Buy & Hold',
        color='salmon',
        alpha=0.85
    )
    axes[0].set_xticks(list(x))
    axes[0].set_xticklabels(summary_df.index, rotation=45, ha='right')
    axes[0].axhline(0, color='black', linewidth=0.8)
    axes[0].set_ylabel('Return (%)')
    axes[0].set_title('Return: Strategy vs Buy & Hold')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3, axis='y')

    colors = ['#2ecc71' if s > 1.0 else '#e74c3c' for s in
summary_df['Sharpe']]
    axes[1].bar(list(x), summary_df['Sharpe'], color=colors,
alpha=0.85)
    axes[1].set_xticks(list(x))
    axes[1].set_xticklabels(summary_df.index, rotation=45, ha='right')
    axes[1].axhline(1.0, color='black', linestyle='--', linewidth=1.2,
label='Sharpe = 1.0')
    axes[1].set_ylabel('Sharpe Ratio')
    axes[1].set_title('Sharpe Ratio (green = above 1.0)')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.show()

    return summary_df

summary_xgb = plot_backtest_summary(result_xgboost,
model_name="xgboost")


==========================================================================
==
```

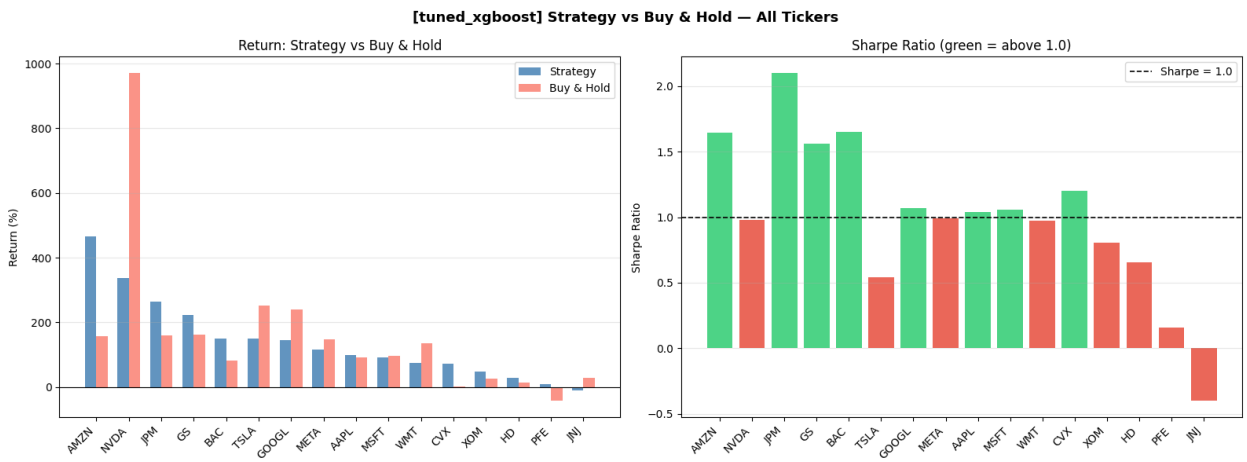
```
  [xgboost] ALL-TICKER BACKTEST SUMMARY
==========================================================================
==
         Return % Buy & Hold %    Sharpe   Max DD % Win Rate % #
Trades Avg Trade % Profit Factor Beat B&H Sharpe OK
AMZN   203.253456    156.79734  1.376968 -12.417191      68.75
32    3.545884      9.474937      Yes       Yes
JPM     87.888331    159.46862  1.537163 -12.434587  76.923077
13    5.019623      9.036663       No       Yes
WMT     83.638907   135.285236  1.296898  -8.958792  51.724138
29    2.132661      4.070552       No       Yes
META    64.325493   147.345035  0.814519 -21.946525  78.571429
14    3.650803      4.075347       No        No
GOOGL   54.758563    238.89336  0.681668 -20.436502  54.098361
61    0.724562      1.883578       No        No
GS      51.390302   162.576763  0.859505 -15.446338  68.421053
19    2.252897      2.932556       No        No
MSFT    33.261709    97.669472  0.614892 -23.164363       40.0
15    1.992208      2.996995       No        No
AAPL    19.501393    91.847009  0.313801  -17.62387      56.25
16    1.122848      1.987247       No        No
CVX     16.474544     1.777953  0.346842 -15.378232  54.166667
24    0.692681      1.721479      Yes        No
HD       8.371306    13.970326  0.242815 -15.079527       44.0
25    0.330862      1.399179       No        No
XOM      4.712554    26.437588  0.116592 -17.178457  60.606061
33    0.143097      1.166741       No        No
NVDA     3.902332   970.588746  0.041247  -49.01935  49.090909
55    0.066339      1.138959       No        No
BAC     -3.433306     82.72389 -0.103542 -23.940058  38.461538
13   -0.269013      0.940085       No        No
JNJ    -16.280464    27.457548 -0.736357 -22.720506     40.625
32   -0.560733      0.566452       No        No
TSLA   -24.088542   252.873115 -0.206778 -57.821888  46.428571
56   -0.511119      1.055844       No        No
PFE    -27.201877   -43.009033 -0.716496 -38.749987  44.827586
58   -0.548596      0.679835      Yes        No

  Tickers beating Buy & Hold : 3 / 16
  Tickers with Sharpe > 1.0  : 3 / 16
  Avg strategy return        : 35.03%
  Avg buy & hold return      : 157.67%
```




```
summary_tuned_xgb = plot_backtest_summary(tuned_xgboost,
model_name="tuned_xgboost")
```

```
==========================================================================
==
  [tuned_xgboost] ALL-TICKER BACKTEST SUMMARY
==========================================================================
==
         Return % Buy & Hold %    Sharpe   Max DD % Win Rate % #
Trades Avg Trade % Profit Factor Beat B&H Sharpe OK
AMZN   464.690499    156.79734  1.645989 -12.426247      100.0
15   11.796158           NaN      Yes       Yes
NVDA   336.760474   970.588746  0.979182  -27.41336  70.731707
41    3.503844      3.831107       No        No
JPM    264.327904    159.46862  2.100655  -5.878599  83.333333
12   10.853123     52.555051      Yes       Yes
GS     222.165355   162.576763  1.560256 -11.899466  63.636364
11    9.796929      18.78932      Yes       Yes
BAC    150.827288     82.72389  1.653466  -8.184932  72.727273
11    7.583456     11.638152      Yes       Yes
TSLA   150.671096   252.873115  0.539867 -44.260117  46.666667
30    3.123369        2.0625       No        No
GOOGL  143.929416    238.89336   1.07132 -20.255344  55.882353
34     2.67748      2.898971       No       Yes
META   116.471314   147.345035  0.989219 -20.323432  61.538462
13    5.524671      7.638559       No        No
AAPL    97.773763    91.847009   1.04034 -14.385543  86.666667
15    4.085538      7.388231      Yes       Yes
MSFT     90.91538    97.669472  1.056401 -15.244314  69.230769
13    5.143726      5.634943       No       Yes
WMT     73.956434   135.285236  0.976334 -24.743236  57.692308
26    1.785304      3.357349       No        No
CVX     71.147926     1.777953  1.203552 -11.292011  66.666667
18    3.053666      7.278122      Yes       Yes
```

```
XOM      48.998814    26.437588  0.804075  -13.55871  68.181818
22    1.757625      3.705139      Yes        No
HD       28.543085    13.970326  0.652894 -17.939954  68.421053
19    1.368105      2.536116      Yes        No
PFE       8.925402   -43.009033  0.160223 -30.421752  59.090909
22    0.388874      1.300958      Yes        No
JNJ      -11.43668    27.457548  -0.40301 -23.162634  31.818182
22   -0.557473      0.657069       No        No

  Tickers beating Buy & Hold : 9 / 16
  Tickers with Sharpe > 1.0  : 8 / 16
  Avg strategy return        : 141.17%
  Avg buy & hold return      : 157.67%
```



#Saving the best model

```
import joblib

SAVE_MODEL_NAME = OPTUNA_MODEL_NAME

joblib.dump(
    {
        'model': tuned_model,
        'scaler': tuned_scaler,
        'features': tuned_features
    },
    f'{SAVE_MODEL_NAME}.pkl'
)

print(f'Saved: {SAVE_MODEL_NAME}.pkl')
```

```
Saved: xgboost_tuned.pkl
```

#LSTM Model

```
# LSTM config — adjust freely
LSTM_MODEL_NAME  = 'lstm_v1'   # used for saving files
LSTM_WINDOW      = 30          # how many past days to feed as a
sequence
LSTM_HIDDEN      = 128         # number of hidden units per LSTM layer
LSTM_LAYERS      = 2           # number of stacked LSTM layers
LSTM_DROPOUT     = 0.3         # dropout between layers
LSTM_EPOCHS      = 50
LSTM_BATCH_SIZE  = 256
LSTM_LR          = 1e-3        # learning rate
LSTM_FRACTION    = TEST_FRACTION

def make_sequences(X_arr, y_arr, window):

    Xs, ys = [], []
    for i in range(window, len(X_arr)):
        Xs.append(X_arr[i - window : i])   # past 'window' days
        ys.append(y_arr[i])                 # label for day i
    return np.array(Xs, dtype=np.float32), np.array(ys,
dtype=np.int64)


class SequenceDataset(Dataset):
    def __init__(self, X_seq, y_seq):
        self.X = torch.tensor(X_seq)
        self.y = torch.tensor(y_seq)
    def __len__(self):
        return len(self.y)
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]


print('make_sequences and SequenceDataset defined')
```

```
make_sequences and SequenceDataset defined
```

```
class LSTMClassifier(nn.Module):
    def __init__(self, input_size, hidden_size=128, num_layers=2,
                 dropout=0.3, num_classes=3):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size  = input_size,
            hidden_size = hidden_size,
            num_layers  = num_layers,
            dropout     = dropout if num_layers > 1 else 0.0,
            batch_first = True,     # input shape: (batch, seq_len,
features)
        )
        self.dropout = nn.Dropout(dropout)
        self.fc      = nn.Linear(hidden_size, num_classes)
```

```
    def forward(self, x):
        # x shape: (batch, seq_len, input_size)
        out, _ = self.lstm(x)
        out     = self.dropout(out[:, -1, :])   # take last timestep
only
        return self.fc(out)                      # (batch, 3)


print('LSTMClassifier defined')
```

```
LSTMClassifier defined
```

```
def train_sequence_model(combined_df, model_class,
                         model_name='lstm_v1',
                         window=30, hidden=128, layers=2,
dropout=0.3,
                         epochs=50, batch_size=256, lr=1e-3,
                         test_fraction=0.2):
    """
    General trainer for LSTM or GRU — pass model_class=LSTMClassifier
or GRUClassifier.
    Returns: pytorch_model, scaler, feature_cols, train_history
    """
    available = [c for c in FEATURE_COLS if c in combined_df.columns]
    X_raw = combined_df[available].values.astype(np.float32)
    y_raw = combined_df['label'].values.astype(np.int64)

    # Train / test split
    split     = int(len(X_raw) * (1 - test_fraction))
    X_tr_raw  = X_raw[:split]
    X_te_raw  = X_raw[split:]
    y_tr_raw  = y_raw[:split]
    y_te_raw  = y_raw[split:]

    # Scale — fit on train only
    sc = StandardScaler()
    X_tr_scaled = sc.fit_transform(X_tr_raw)
    X_te_scaled = sc.transform(X_te_raw)

    # Build sequences
    X_tr_seq, y_tr_seq = make_sequences(X_tr_scaled, y_tr_raw, window)
    X_te_seq, y_te_seq = make_sequences(X_te_scaled, y_te_raw, window)

    train_ds = SequenceDataset(X_tr_seq, y_tr_seq)
    val_ds   = SequenceDataset(X_te_seq, y_te_seq)
    train_dl = DataLoader(train_ds, batch_size=batch_size,
shuffle=False)
    val_dl   = DataLoader(val_ds,   batch_size=batch_size,
shuffle=False)
```

```
    # Class weights for imbalance
    counts   = np.bincount(y_tr_seq)
    weights  = torch.tensor(1.0 / counts,
dtype=torch.float32).to(DEVICE)

    # Model
    net = model_class(
        input_size  = len(available),
        hidden_size = hidden,
        num_layers  = layers,
        dropout     = dropout,
    ).to(DEVICE)

    optimizer = torch.optim.Adam(net.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss(weight=weights)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, patience=5, factor=0.5)

    # Training loop
    history = {'train_loss': [], 'val_loss': [], 'val_acc': []}
    best_val_loss = float('inf')
    best_state    = None

    print(f'[{model_name}] Training on {len(X_tr_seq):,} sequences  '
          f'| val on {len(X_te_seq):,}  |  window={window}  '
          f'epochs={epochs}  device={DEVICE}\n')

    for epoch in range(epochs):
        # Train
        net.train()
        tr_loss = 0
        for xb, yb in train_dl:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            optimizer.zero_grad()
            loss = criterion(net(xb), yb)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(net.parameters(), 1.0)
            optimizer.step()
            tr_loss += loss.item() * len(yb)
        tr_loss /= len(train_ds)

        # Validate
        net.eval()
        vl_loss, correct = 0, 0
        with torch.no_grad():
            for xb, yb in val_dl:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)
                logits = net(xb)
                vl_loss += criterion(logits, yb).item() * len(yb)
                correct += (logits.argmax(1) == yb).sum().item()
```

```
        vl_loss /= len(val_ds)
        vl_acc   = correct / len(val_ds)

        scheduler.step(vl_loss)
        history['train_loss'].append(tr_loss)
        history['val_loss'].append(vl_loss)
        history['val_acc'].append(vl_acc)

        # Save best weights
        if vl_loss < best_val_loss:
            best_val_loss = vl_loss
            best_state    = {k: v.cpu().clone() for k, v in
net.state_dict().items()}

        if (epoch + 1) % 10 == 0:
            print(f'  Epoch {epoch+1:3d}/{epochs}  '
                  f'train_loss: {tr_loss:.4f}  '
                  f'val_loss: {vl_loss:.4f}  '
                  f'val_acc: {vl_acc:.3f}')

    # Load best weights
    net.load_state_dict(best_state)
    net.eval()
    print(f'\n [{model_name}] training complete  '
          f'(best val_loss: {best_val_loss:.4f})')

    # Plot training curves
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    fig.suptitle(f'[{model_name}] Training Curves')
    axes[0].plot(history['train_loss'], label='Train Loss')
    axes[0].plot(history['val_loss'],   label='Val Loss')
    axes[0].set_title('Loss')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)
    axes[1].plot(history['val_acc'], color='green', label='Val
Accuracy')
    axes[1].set_title('Validation Accuracy')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

    return net, sc, available, history


print('train_sequence_model defined')
```

```
train_sequence_model defined
```

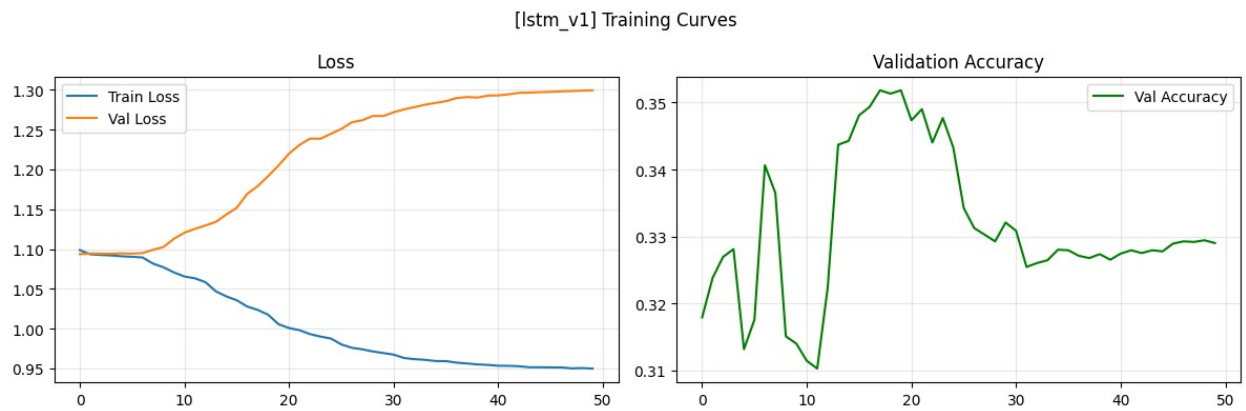
```python
class SequenceModelWrapper:
    def __init__(self, net, window, device=DEVICE):
        self.net    = net
        self.window = window
        self.device = device
        # These mirror sklearn attributes used by evaluate_model
        self.feature_importances_ = None

    def _to_sequences(self, X_arr):
        Xs = []
        for i in range(self.window, len(X_arr) + 1):
            Xs.append(X_arr[i - self.window : i])
        return np.array(Xs, dtype=np.float32)

    def predict_proba(self, X_arr):
        self.net.eval()
        seqs   = self._to_sequences(X_arr)
        tensor = torch.tensor(seqs).to(self.device)
        with torch.no_grad():
            logits = self.net(tensor)
            proba  = torch.softmax(logits, dim=1).cpu().numpy()
        # Pad front with zeros so length matches original X_arr
        pad   = np.zeros((self.window - 1, 3))
        return np.vstack([pad, proba])

    def predict(self, X_arr):
        return self.predict_proba(X_arr).argmax(axis=1)


print('SequenceModelWrapper defined')
```

```
SequenceModelWrapper defined
```

```python
# Train LSTM
lstm_net, lstm_scaler, lstm_features, lstm_history =
train_sequence_model(
    combined,
    model_class = LSTMClassifier,
    model_name  = LSTM_MODEL_NAME,
    window      = LSTM_WINDOW,
    hidden      = LSTM_HIDDEN,
    layers      = LSTM_LAYERS,
    dropout     = LSTM_DROPOUT,
    epochs      = LSTM_EPOCHS,
    batch_size  = LSTM_BATCH_SIZE,
    lr          = LSTM_LR,
    test_fraction = LSTM_FRACTION,
)
```

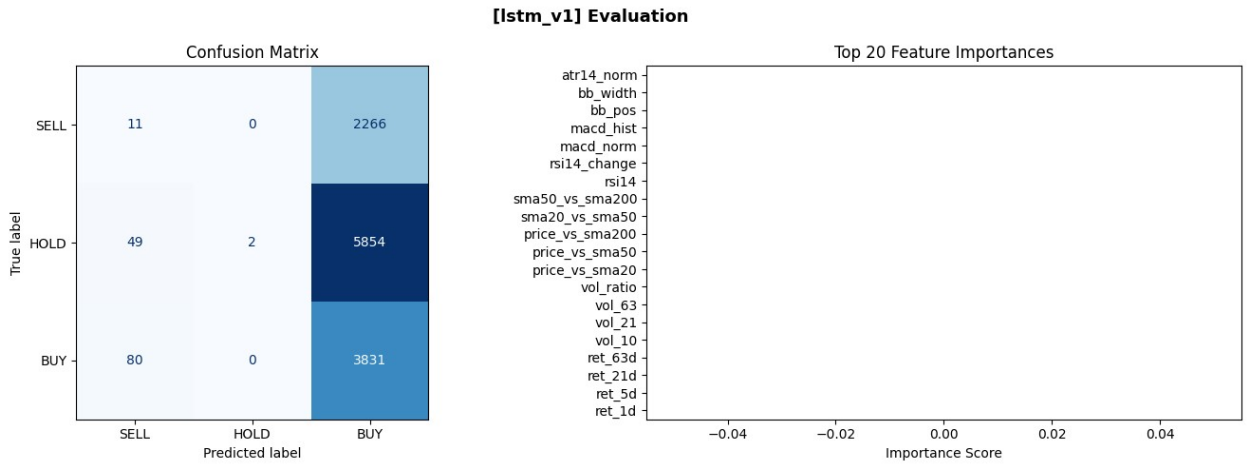
```
# Wrap so it works with evaluate_model() and run_backtest()
lstm_model = SequenceModelWrapper(lstm_net, window=LSTM_WINDOW)
```

```
[lstm_v1] Training on 48,340 sequences  | val on 12,063  |  window=30
epochs=50  device=cpu

  Epoch  10/50  train_loss: 1.0705  val_loss: 1.1129  val_acc: 0.314
  Epoch  20/50  train_loss: 1.0054  val_loss: 1.2051  val_acc: 0.352
  Epoch  30/50  train_loss: 0.9692  val_loss: 1.2672  val_acc: 0.332
  Epoch  40/50  train_loss: 0.9542  val_loss: 1.2929  val_acc: 0.327
  Epoch  50/50  train_loss: 0.9496  val_loss: 1.2996  val_acc: 0.329

 [lstm_v1] training complete  (best val_loss: 1.0936)
```




```
# Evaluate — same function as XGBoost, just pass the wrapper
lstm_preds = evaluate_model(
    lstm_model, lstm_scaler, lstm_features, combined,
    model_name    = LSTM_MODEL_NAME,
    test_fraction = LSTM_FRACTION,
)
```

```
[lstm_v1] Holdout Evaluation (last 20% of combined data)

              precision    recall  f1-score   support

        SELL       0.08      0.00      0.01      2277
        HOLD       1.00      0.00      0.00      5905
         BUY       0.32      0.98      0.48      3911

    accuracy                           0.32     12093
   macro avg       0.47      0.33      0.16     12093
weighted avg       0.61      0.32      0.16     12093
```




```
all_stats = {}
total_invested = 0

for ticker in stock_data.keys():
    stats, bt = run_backtest(
        lstm_model, lstm_scaler, lstm_features, combined, stock_data,
        ticker=ticker
    )
    if stats is not None:
        all_stats[ticker] = stats
        total_invested += BT_CASH

# Summary
total_final   = sum(s['Equity Final [$]'] for s in all_stats.values())
total_profit  = total_final - total_invested
total_return  = (total_profit / total_invested) * 100 if
total_invested > 0 else 0

# Build formatted summary
summary_text = (
    f"\n{'='*40}\n"
    f"  OVERALL PORTFOLIO SUMMARY\n"
    f"{'='*40}\n"
    f"  Tickers traded       {len(all_stats)}\n"
    f"  Total invested       ${total_invested:,.0f}\n"
    f"  Total final equity   ${total_final:,.2f}\n"
    f"  Total profit/loss    ${total_profit:,.2f}\n"
    f"  Overall return       {total_return:.2f}%\n"
)

result_lstm = {
    "tickers_traded": len(all_stats),
    "total_invested": total_invested,
    "total_final_equity": total_final,
    "total_profit_loss": total_profit,
```

```
    "overall_return_percent": total_return,
    "per_ticker_stats": all_stats,
    "summary": summary_text
}

print(result_lstm["summary"])
```

{"model_id":"aee0b47982d6434eaffb1941fc304a5b","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — AAPL from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $19,204.41
  Strategy return          92.04%
  Buy & Hold return        91.85%
  Max drawdown             -33.16%
  Sharpe ratio             0.763
  Win rate                 nan%
  No. of Trades            0
  Avg trade return         nan%
  Profit factor            nan
  Calmar ratio             0.723

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"c0bea92c235f45c8b0eec88553c8908a","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — MSFT from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $20,142.47
  Strategy return          101.42%
  Buy & Hold return        97.67%
  Max drawdown             -23.43%
  Sharpe ratio             0.919
  Win rate                 100.0%
  No. of Trades            1
  Avg trade return         58.92%
  Profit factor            nan
  Calmar ratio             1.107

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"18b9b79a811543359b6b817cc9ea643b","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — GOOGL from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $28,674.49
  Strategy return          186.74%
  Buy & Hold return        238.89%
  Max drawdown             -30.17%
  Sharpe ratio             0.981
  Win rate                 100.0%
  No. of Trades            3
  Avg trade return         39.01%
  Profit factor            nan
  Calmar ratio             1.375

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"13ba8702d8414b57bd1a0ea14aee6d4d","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — AMZN from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $24,083.45
  Strategy return          140.83%
  Buy & Hold return        156.80%
  Max drawdown             -29.44%
  Sharpe ratio             0.794
  Win rate                 100.0%
  No. of Trades            1
  Avg trade return         87.90%
  Profit factor            nan
  Calmar ratio             1.141

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"4e607f764efc44bca9ac96d8445dea8f","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — META from 2023-06-06 to 2025-12-30
================================================================
```

```
  Starting cash             $10,000
  Final equity              $21,659.82
  Strategy return           116.60%
  Buy & Hold return         147.35%
  Max drawdown              -31.73%
  Sharpe ratio              0.740
  Win rate                  100.0%
  No. of Trades             2
  Avg trade return          35.23%
  Profit factor             nan
  Calmar ratio              1.111

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"f9f555bf8efe478f8167a1d1d4b2c3ae","version_major":2,"version_minor":0}

```
==============================================================
  BACKTESTING — JPM from 2022-12-12 to 2025-12-30
==============================================================
  Starting cash             $10,000
  Final equity              $24,842.93
  Strategy return           148.43%
  Buy & Hold return         159.47%
  Max drawdown              -24.31%
  Sharpe ratio              1.138
  Win rate                  100.0%
  No. of Trades             1
  Avg trade return          73.80%
  Profit factor             nan
  Calmar ratio              1.438

  Beat Buy&Hold: No
  Sharpe > 1.0:  Yes
```

{"model_id":"878ff7d64e2c4eb1bbaf698d6380a3d5","version_major":2,"version_minor":0}

```
==============================================================
  BACKTESTING — BAC from 2022-12-12 to 2025-12-30
==============================================================
  Starting cash             $10,000
  Final equity              $17,438.16
  Strategy return           74.38%
  Buy & Hold return         82.72%
  Max drawdown              -29.93%
  Sharpe ratio              0.660
  Win rate                  100.0%
```

```
  No. of Trades            1
  Avg trade return         9.63%
  Profit factor            nan
  Calmar ratio             0.672

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"4df7e3fad3c54d02a085aed4513e6451","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — GS from 2022-12-12 to 2025-12-30
============================================================
  Starting cash            $10,000
  Final equity             $26,629.59
  Strategy return          166.30%
  Buy & Hold return        162.58%
  Max drawdown             -30.53%
  Sharpe ratio             1.042
  Win rate                 nan%
  No. of Trades            0
  Avg trade return         nan%
  Profit factor            nan
  Calmar ratio             1.247

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"78b05915af5d4795be3ee33e9effd399","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — XOM from 2022-12-12 to 2025-12-30
============================================================
  Starting cash            $10,000
  Final equity             $11,741.87
  Strategy return          17.42%
  Buy & Hold return        26.44%
  Max drawdown             -18.55%
  Sharpe ratio             0.238
  Win rate                 0.0%
  No. of Trades            1
  Avg trade return         -1.41%
  Profit factor            N/A
  Calmar ratio             0.293

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"03d883890a894fda9f7cbb245d0ade2e","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — CVX from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $9,864.87
  Strategy return          -1.35%
  Buy & Hold return        1.78%
  Max drawdown             -22.00%
  Sharpe ratio             -0.021
  Win rate                 0.0%
  No. of Trades            1
  Avg trade return         -15.15%
  Profit factor            N/A
  Calmar ratio             -0.020

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"1fa046e731954efc8706c51c1dbbcd26","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — JNJ from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $11,845.83
  Strategy return          18.46%
  Buy & Hold return        27.46%
  Max drawdown             -20.91%
  Sharpe ratio             0.318
  Win rate                 0.0%
  No. of Trades            2
  Avg trade return         -6.41%
  Profit factor            N/A
  Calmar ratio             0.274

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"4a05e4de0e41427ba724539f61c38f5e","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — PFE from 2022-12-12 to 2025-12-30
================================================================
```

```
  Starting cash             $10,000
  Final equity              $6,488.89
  Strategy return           -35.11%
  Buy & Hold return         -43.01%
  Max drawdown              -46.71%
  Sharpe ratio              -0.643
  Win rate                  0.0%
  No. of Trades             2
  Avg trade return          -27.11%
  Profit factor             N/A
  Calmar ratio              -0.284

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"e144c07674bb46f8b6722996fe063fb0","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — WMT from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $24,216.67
  Strategy return           142.17%
  Buy & Hold return         135.29%
  Max drawdown              -21.89%
  Sharpe ratio              1.286
  Win rate                  nan%
  No. of Trades             0
  Avg trade return          nan%
  Profit factor             nan
  Calmar ratio              1.545

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  Yes
```

{"model_id":"6dcd2846d12649e6b2ddb16d4c32e9a5","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — HD from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $11,837.19
  Strategy return           18.37%
  Buy & Hold return         13.97%
  Max drawdown              -21.58%
  Sharpe ratio              0.254
  Win rate                  nan%
```

```
  No. of Trades             0
  Avg trade return          nan%
  Profit factor             nan
  Calmar ratio              0.265

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"9be025a069d24ffeb94fc79a3ff71a83","version_major":2,"version_minor":0}

```
==============================================================
  BACKTESTING — TSLA from 2023-01-18 to 2025-12-30
==============================================================
  Starting cash             $10,000
  Final equity              $19,117.95
  Strategy return           91.18%
  Buy & Hold return         252.87%
  Max drawdown              -62.41%
  Sharpe ratio              0.326
  Win rate                  50.0%
  No. of Trades             2
  Avg trade return          4.20%
  Profit factor             2.43
  Calmar ratio              0.395

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"b1a72afa293547bcbb7c6e2d194782a3","version_major":2,"version_minor":0}

```
==============================================================
  BACKTESTING — NVDA from 2022-12-12 to 2025-12-30
==============================================================
  Starting cash             $10,000
  Final equity              $82,359.26
  Strategy return           723.59%
  Buy & Hold return         970.59%
  Max drawdown              -36.86%
  Sharpe ratio              0.983
  Win rate                  100.0%
  No. of Trades             4
  Avg trade return          52.19%
  Profit factor             nan
  Calmar ratio              2.721

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

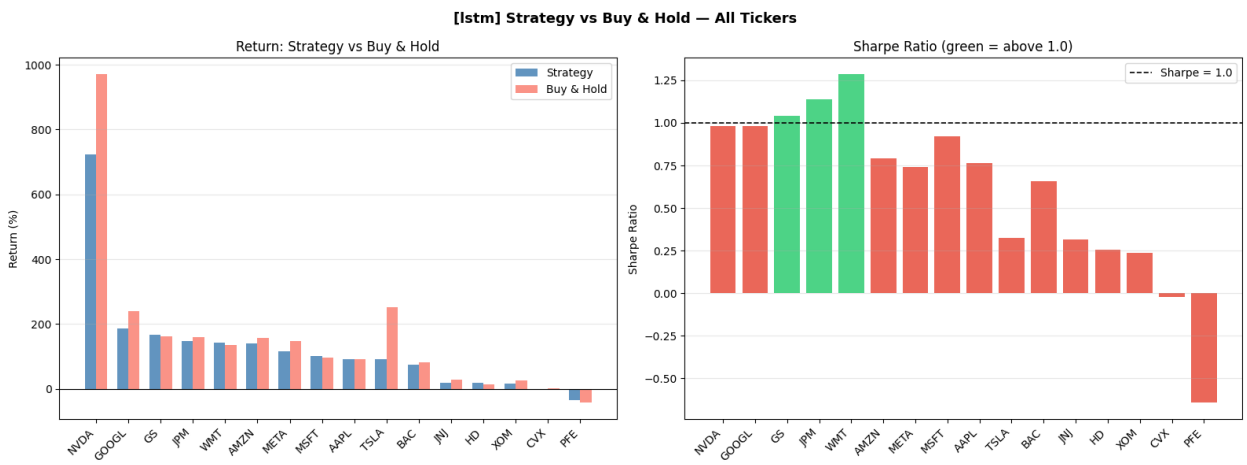
```
========================================
  OVERALL PORTFOLIO SUMMARY
========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $360,147.86
  Total profit/loss    $200,147.86
  Overall return       125.09%
```

LSTM Backtest Summary

```
summary_lstm = plot_backtest_summary(result_lstm, model_name="lstm")
```

```
========================================================================
==
  [lstm] ALL-TICKER BACKTEST SUMMARY
========================================================================
==
         Return % Buy & Hold %    Sharpe   Max DD % Win Rate % #
Trades Avg Trade % Profit Factor Beat B&H Sharpe OK
NVDA   723.592615   970.588746  0.983261 -36.862106      100.0
4   52.186555           NaN       No        No
GOOGL  186.744886    238.89336  0.980993 -30.165573      100.0
3   39.005152           NaN       No        No
GS     166.295933   162.576763  1.041559 -30.529391        NaN
0         NaN           NaN      Yes       Yes
JPM    148.429341    159.46862    1.1385 -24.305845      100.0
1   73.804908           NaN       No       Yes
WMT     142.16665   135.285236  1.285514 -21.887186        NaN
0         NaN           NaN      Yes       Yes
AMZN   140.834477    156.79734  0.793873 -29.439026      100.0
1   87.895935           NaN       No        No
META   116.598196   147.345035  0.739929 -31.731568      100.0
2   35.233438           NaN       No        No
MSFT   101.424725    97.669472  0.919197 -23.433472      100.0
1   58.920745           NaN      Yes        No
AAPL    92.044112    91.847009  0.763128 -33.164328        NaN
0         NaN           NaN      Yes        No
TSLA     91.17951   252.873115  0.326092 -62.412702       50.0
2    4.202513      2.433372       No        No
BAC     74.381575     82.72389  0.659575 -29.930538      100.0
1    9.629582           NaN       No        No
JNJ     18.458323    27.457548  0.317618 -20.911098        0.0
2   -6.406821           0.0       No        No
HD      18.371915    13.970326  0.254315 -21.579859        NaN
0         NaN           NaN      Yes        No
XOM     17.418746    26.437588  0.237803 -18.551642        0.0
```

```
1   -1.408562           0.0       No        No
CVX     -1.351293     1.777953  -0.02145 -21.999677        0.0
1  -15.145568           0.0       No        No
PFE    -35.111066   -43.009033 -0.643191 -46.706885        0.0
2   -27.11293           0.0      Yes        No

  Tickers beating Buy & Hold : 6 / 16
  Tickers with Sharpe > 1.0  : 3 / 16
  Avg strategy return        : 125.09%
  Avg buy & hold return      : 157.67%
```



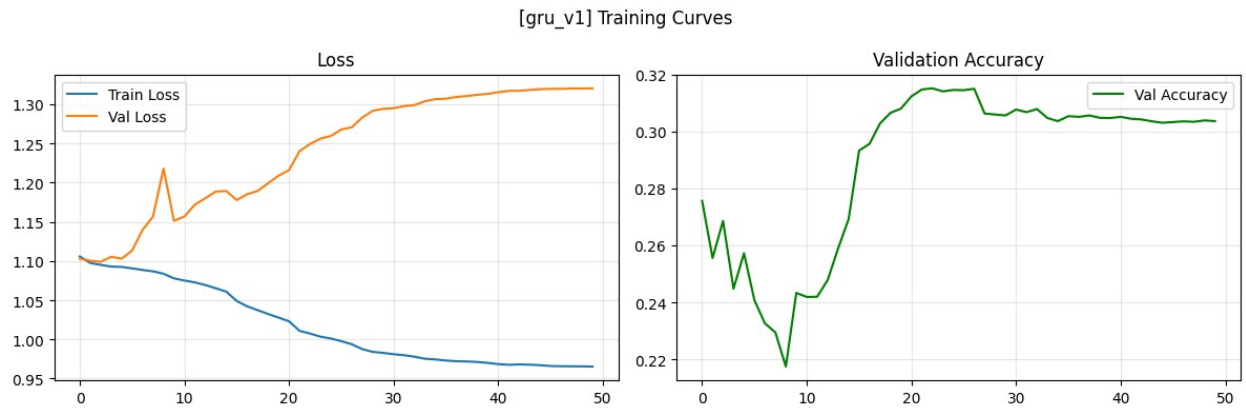
#GRU model

```python
class GRUClassifier(nn.Module):
    def __init__(self, input_size, hidden_size=128, num_layers=2,
                 dropout=0.3, num_classes=3):
        super().__init__()
        self.gru = nn.GRU(
            input_size  = input_size,
            hidden_size = hidden_size,
            num_layers  = num_layers,
            dropout     = dropout if num_layers > 1 else 0.0,
            batch_first = True,
        )
        self.dropout = nn.Dropout(dropout)
        self.fc      = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        out, _ = self.gru(x)
        out     = self.dropout(out[:, -1, :])   # last timestep
        return self.fc(out)


print('GRUClassifier defined')
```

```
GRUClassifier defined
```

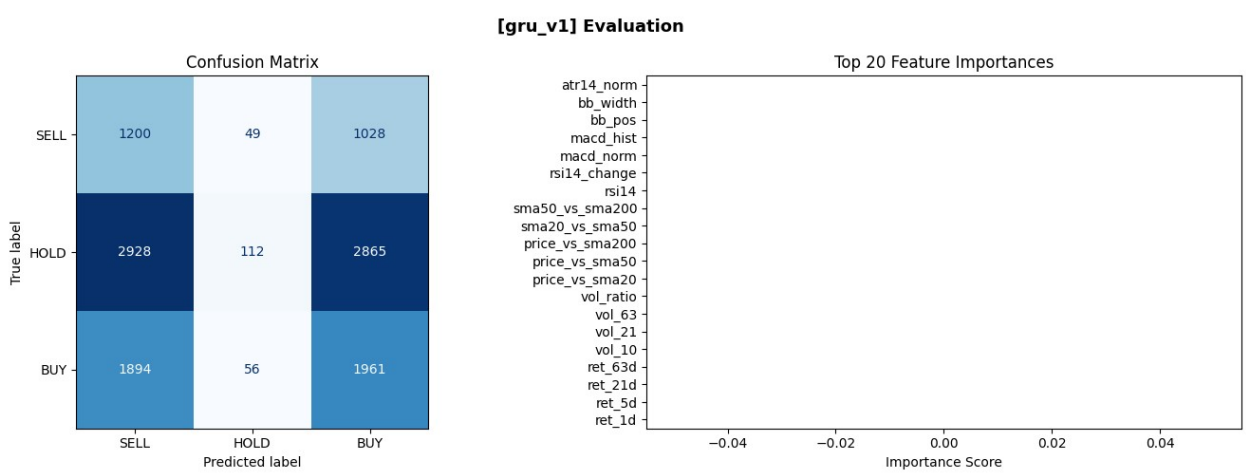
```
GRU_MODEL_NAME  = 'gru_v1'
GRU_WINDOW      = 30
GRU_HIDDEN      = 128
GRU_LAYERS      = 2
GRU_DROPOUT     = 0.3
GRU_EPOCHS      = 50
GRU_BATCH_SIZE  = 256
GRU_LR          = 1e-3
GRU_FRACTION    = TEST_FRACTION

gru_net, gru_scaler, gru_features, gru_history = train_sequence_model(
    combined,
    model_class = GRUClassifier,
    model_name  = GRU_MODEL_NAME,
    window      = GRU_WINDOW,
    hidden      = GRU_HIDDEN,
    layers      = GRU_LAYERS,
    dropout     = GRU_DROPOUT,
    epochs      = GRU_EPOCHS,
    batch_size  = GRU_BATCH_SIZE,
    lr          = GRU_LR,
    test_fraction = GRU_FRACTION,
)

gru_model = SequenceModelWrapper(gru_net, window=GRU_WINDOW)
```

```
[gru_v1] Training on 48,340 sequences  | val on 12,063  |  window=30
epochs=50  device=cpu

  Epoch  10/50  train_loss: 1.0778  val_loss: 1.1511  val_acc: 0.243
  Epoch  20/50  train_loss: 1.0278  val_loss: 1.2088  val_acc: 0.308
  Epoch  30/50  train_loss: 0.9828  val_loss: 1.2939  val_acc: 0.306
  Epoch  40/50  train_loss: 0.9700  val_loss: 1.3131  val_acc: 0.305
  Epoch  50/50  train_loss: 0.9653  val_loss: 1.3201  val_acc: 0.304

 [gru_v1] training complete  (best val_loss: 1.0989)
```




```
gru_preds = evaluate_model(
    gru_model, gru_scaler, gru_features, combined,
    model_name    = GRU_MODEL_NAME,
    test_fraction = GRU_FRACTION,
)
```

```
[gru_v1] Holdout Evaluation (last 20% of combined data)

              precision    recall  f1-score   support

        SELL       0.20      0.53      0.29      2277
        HOLD       0.52      0.02      0.04      5905
         BUY       0.33      0.50      0.40      3911

    accuracy                           0.27     12093
   macro avg       0.35      0.35      0.24     12093
weighted avg       0.40      0.27      0.20     12093
```




```
all_stats = {}
total_invested = 0
```

```python
for ticker in stock_data.keys():
    stats, bt = run_backtest(
        gru_model, gru_scaler, gru_features, combined, stock_data,
        ticker=ticker
    )
    if stats is not None:
        all_stats[ticker] = stats
        total_invested += BT_CASH

# Summary
total_final   = sum(s['Equity Final [$]'] for s in all_stats.values())
total_profit  = total_final - total_invested
total_return  = (total_profit / total_invested) * 100 if
total_invested > 0 else 0

# Build formatted summary
summary_text = (
    f"\n{'='*40}\n"
    f"  GRU OVERALL PORTFOLIO SUMMARY\n"
    f"{'='*40}\n"
    f"  Tickers traded       {len(all_stats)}\n"
    f"  Total invested       ${total_invested:,.0f}\n"
    f"  Total final equity   ${total_final:,.2f}\n"
    f"  Total profit/loss    ${total_profit:,.2f}\n"
    f"  Overall return       {total_return:.2f}%\n"
)

result_gru = {
    "tickers_traded": len(all_stats),
    "total_invested": total_invested,
    "total_final_equity": total_final,
    "total_profit_loss": total_profit,
    "overall_return_percent": total_return,
    "per_ticker_stats": all_stats,
    "summary": summary_text
}

print(result_gru["summary"])
```

{"model_id":"6c9e78791fb641f0b29bce6f27ce4ae0","version_major":2,"version_minor":0}

```
==========================================================
  BACKTESTING — AAPL from 2022-12-12 to 2025-12-30
==========================================================
  Starting cash            $10,000
  Final equity             $10,919.51
  Strategy return          9.20%
```

```
  Buy & Hold return         91.85%
  Max drawdown              -27.03%
  Sharpe ratio              0.165
  Win rate                  59.3%
  No. of Trades             27
  Avg trade return          0.29%
  Profit factor             1.20
  Calmar ratio              0.109

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"43870d2cd88c482da5439b8d6ac8639c","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — MSFT from 2022-12-12 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $15,675.87
  Strategy return           56.76%
  Buy & Hold return         97.67%
  Max drawdown              -12.70%
  Sharpe ratio              0.825
  Win rate                  66.7%
  No. of Trades             24
  Avg trade return          1.86%
  Profit factor             2.64
  Calmar ratio              1.257

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"ff3b67a49aea47bf80fba51104ba82a8","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — GOOGL from 2022-12-12 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $13,904.15
  Strategy return           39.04%
  Buy & Hold return         238.89%
  Max drawdown              -29.78%
  Sharpe ratio              0.477
  Win rate                  60.0%
  No. of Trades             25
  Avg trade return          1.20%
  Profit factor             1.66
```

```
  Calmar ratio              0.385

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"68add731554b406fb9b39f3760358346","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — AMZN from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $14,788.31
  Strategy return           47.88%
  Buy & Hold return         156.80%
  Max drawdown              -28.00%
  Sharpe ratio              0.506
  Win rate                  53.6%
  No. of Trades             28
  Avg trade return          1.16%
  Profit factor             1.68
  Calmar ratio              0.491

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"ef78d40e63d640bda83bc4e163b4bd9a","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — META from 2023-06-06 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $16,558.55
  Strategy return           65.59%
  Buy & Hold return         147.35%
  Max drawdown              -18.85%
  Sharpe ratio              0.739
  Win rate                  63.2%
  No. of Trades             19
  Avg trade return          2.66%
  Profit factor             2.61
  Calmar ratio              1.156

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"0d55a293f2f548e496a06fe507130d3a","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — JPM from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $11,984.29
  Strategy return          19.84%
  Buy & Hold return        159.47%
  Max drawdown             -15.27%
  Sharpe ratio             0.328
  Win rate                 60.7%
  No. of Trades            28
  Avg trade return         0.73%
  Profit factor            1.56
  Calmar ratio             0.402

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"a8674b533039428aa3cf7163e240f0ab","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — BAC from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $9,244.84
  Strategy return          -7.55%
  Buy & Hold return        82.72%
  Max drawdown             -31.45%
  Sharpe ratio             -0.141
  Win rate                 66.7%
  No. of Trades            21
  Avg trade return         -0.59%
  Profit factor            0.92
  Calmar ratio             -0.081

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"6395237c5d994a1cb415dfbf2511b326","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — GS from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $13,797.17
  Strategy return          37.97%
```

```
  Buy & Hold return         162.58%
  Max drawdown              -21.66%
  Sharpe ratio              0.496
  Win rate                  58.3%
  No. of Trades             24
  Avg trade return          1.39%
  Profit factor             1.75
  Calmar ratio              0.516

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"7402376903ac4a5599d97c74d8a4f77f","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — XOM from 2022-12-12 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $8,511.75
  Strategy return           -14.88%
  Buy & Hold return         26.44%
  Max drawdown              -25.22%
  Sharpe ratio              -0.319
  Win rate                  47.2%
  No. of Trades             36
  Avg trade return          -0.45%
  Profit factor             0.79
  Calmar ratio              -0.205

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"adf5ac83070a44239919a6bbf4b6c4da","version_major":2,"version_minor":0}

```
============================================================
  BACKTESTING — CVX from 2022-12-12 to 2025-12-30
============================================================
  Starting cash             $10,000
  Final equity              $9,591.67
  Strategy return           -4.08%
  Buy & Hold return         1.78%
  Max drawdown              -17.14%
  Sharpe ratio              -0.082
  Win rate                  50.0%
  No. of Trades             30
  Avg trade return          -0.12%
  Profit factor             0.97
```

```
  Calmar ratio              -0.080

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"808fe33a0ee54c1090b8605dd296cebd","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — JNJ from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $7,505.31
  Strategy return           -24.95%
  Buy & Hold return         27.46%
  Max drawdown              -35.01%
  Sharpe ratio              -0.855
  Win rate                  39.3%
  No. of Trades             28
  Avg trade return          -1.03%
  Profit factor             0.51
  Calmar ratio              -0.258

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"3ee4145b59c5412bbb6209a1f80acaa0","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — PFE from 2022-12-12 to 2025-12-30
================================================================
  Starting cash             $10,000
  Final equity              $8,923.58
  Strategy return           -10.76%
  Buy & Hold return         -43.01%
  Max drawdown              -29.18%
  Sharpe ratio              -0.232
  Win rate                  34.8%
  No. of Trades             23
  Avg trade return          -0.42%
  Profit factor             0.85
  Calmar ratio              -0.126

  Beat Buy&Hold: Yes
  Sharpe > 1.0:  No
```

{"model_id":"b887db3a21df4bc3bfe8f07ec2429831","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — WMT from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $14,683.80
  Strategy return          46.84%
  Buy & Hold return        135.29%
  Max drawdown             -13.18%
  Sharpe ratio             0.821
  Win rate                 66.7%
  No. of Trades            21
  Avg trade return         1.86%
  Profit factor            3.02
  Calmar ratio             1.024

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"55ab2a54a8b649f8b5eafedd076b722f","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — HD from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $7,637.05
  Strategy return          -23.63%
  Buy & Hold return        13.97%
  Max drawdown             -32.17%
  Sharpe ratio             -0.632
  Win rate                 55.6%
  No. of Trades            27
  Avg trade return         -0.90%
  Profit factor            0.63
  Calmar ratio             -0.264

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

{"model_id":"4a9cbb25c8c74595816827aed7eb93c1","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — TSLA from 2023-01-18 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $16,654.86
  Strategy return          66.55%
```

```
  Buy & Hold return        252.87%
  Max drawdown             -53.36%
  Sharpe ratio             0.335
  Win rate                 64.0%
  No. of Trades            25
  Avg trade return         1.96%
  Profit factor            1.73
  Calmar ratio             0.355

  Beat Buy&Hold: No
  Sharpe > 1.0:  No
```

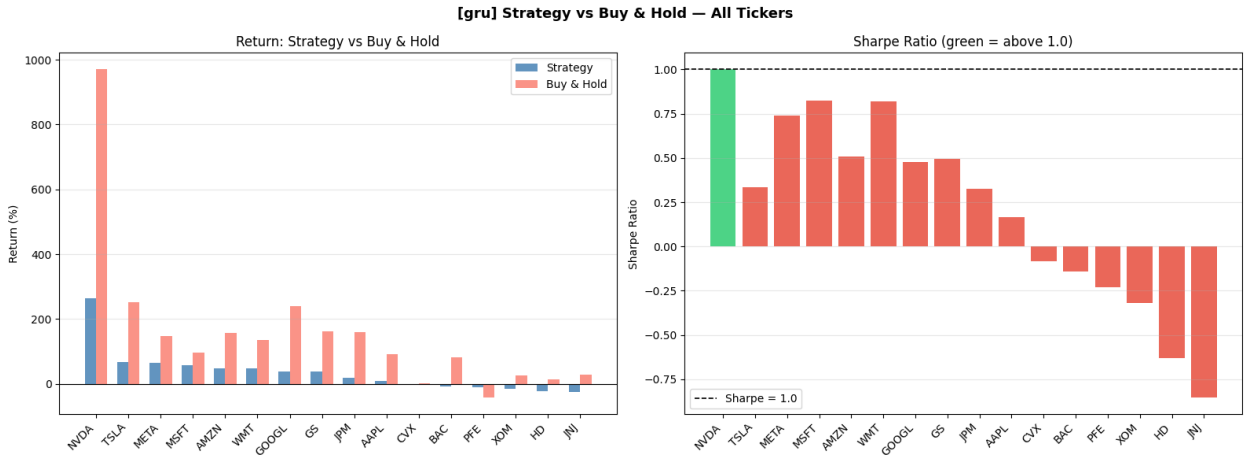
{"model_id":"0874be6dcc7541628d569f2398a073ea","version_major":2,"version_minor":0}

```
================================================================
  BACKTESTING — NVDA from 2022-12-12 to 2025-12-30
================================================================
  Starting cash            $10,000
  Final equity             $36,488.18
  Strategy return          264.88%
  Buy & Hold return        970.59%
  Max drawdown             -21.61%
  Sharpe ratio             1.001
  Win rate                 76.2%
  No. of Trades            21
  Avg trade return         6.39%
  Profit factor            13.10
  Calmar ratio             2.461

  Beat Buy&Hold: No
  Sharpe > 1.0:  Yes

========================================
  GRU OVERALL PORTFOLIO SUMMARY
========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $216,868.90
  Total profit/loss    $56,868.90
  Overall return       35.54%
```

GRU Backtest Summary

```
summary_gru = plot_backtest_summary(result_gru, model_name="gru")


=========================================================================
```

```
==
  [gru] ALL-TICKER BACKTEST SUMMARY
======================================================================
==
         Return % Buy & Hold %    Sharpe   Max DD % Win Rate % #
Trades Avg Trade % Profit Factor Beat B&H Sharpe OK
NVDA   264.881815   970.588746   1.00119 -21.607141  76.190476
21    6.388736     13.100257       No       Yes
TSLA    66.548594   252.873115  0.335184 -53.357797       64.0
25    1.963707      1.731313       No        No
META    65.585548   147.345035  0.739364 -18.845512  63.157895
19    2.660027      2.612431       No        No
MSFT    56.758688    97.669472  0.824884 -12.696952  66.666667
24    1.863371      2.638381       No        No
AMZN    47.883143    156.79734  0.506498  -28.00073  53.571429
28    1.160484       1.67624       No        No
WMT     46.837954   135.285236  0.820778 -13.179161  66.666667
21      1.8593      3.021427       No        No
GOOGL     39.0415    238.89336  0.476759 -29.777492       60.0
25    1.199209      1.660008       No        No
GS      37.971717   162.576763  0.495796 -21.657161  58.333333
24    1.391477        1.7524       No        No
JPM     19.842909    159.46862  0.327635  -15.26753  60.714286
28    0.730342      1.559272       No        No
AAPL      9.19514    91.847009  0.165223 -27.030965  59.259259
27    0.286029      1.202719       No        No
CVX     -4.083273     1.777953 -0.082109 -17.143049       50.0
30   -0.121293      0.972286       No        No
BAC     -7.551583     82.72389 -0.140517 -31.451063  66.666667
21   -0.589495      0.915765       No        No
PFE    -10.764171   -43.009033 -0.231783 -29.179731  34.782609
23   -0.417872      0.850566      Yes        No
XOM    -14.882507    26.437588 -0.319237 -25.221828  47.222222
36   -0.449681      0.789642       No        No
HD     -23.629513    13.970326 -0.631711 -32.166531  55.555556
27   -0.902569      0.631466       No        No
JNJ    -24.946944    27.457548 -0.855106 -35.008027  39.285714
28   -1.027077      0.514861       No        No

  Tickers beating Buy & Hold : 1 / 16
  Tickers with Sharpe > 1.0  : 1 / 16
  Avg strategy return        : 35.54%
  Avg buy & hold return      : 157.67%
```



#Comparing all backtesting result

```
print(result_xgboost["summary"])
print(tuned_xgboost["summary"])
print(result_lstm["summary"])
print(result_gru["summary"])
```

```
==========================================
XGBoost  OVERALL PORTFOLIO SUMMARY
==========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $216,047.47
  Total profit/loss    $56,047.47
  Overall return       35.03%
```

```
==========================================
 XGBOOST TUNED OVERALL PORTFOLIO SUMMARY
==========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $385,866.75
  Total profit/loss    $225,866.75
  Overall return       141.17%
```

```
==========================================
  OVERALL PORTFOLIO SUMMARY
==========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $360,147.86
  Total profit/loss    $200,147.86
  Overall return       125.09%
```

```
==========================================
  GRU OVERALL PORTFOLIO SUMMARY
==========================================
  Tickers traded       16
  Total invested       $160,000
  Total final equity   $216,868.90
  Total profit/loss    $56,868.90
  Overall return       35.54%
```